A Landscape Classification Framework for NP-Hard Problems: From Reconnaissance to Algorithm Selection

NP 難題的曲面分類框架：
從偵察到演算法選擇

Classifying NP-Hard Problems by Operational Structure into Seven Boxes, Mapping Their Loss Landscapes, and Matching Optimal Solvers

tonyrao (饒滙盈)
Independent Researcher
ORCID: 0009-0002-9497-1336

Part of the NSD / MOEH research programme.

Working Paper — Version 7
April 2026

Abstract

We propose a unified classification framework for NP-hard problems based on loss landscape topology. The framework operates in three layers: (1) a seven-box problem taxonomy organised by *operational structure* (selection, ordering, allocation, cutting, colouring, covering, clique-finding, connection, logical satisfaction, path-finding, routing, network design, scheduling, matching, layout, sequential decision), further divided into 24 sub-boxes (21 main sub-boxes with formal objective functions in Boxes 1–6, plus 3 miscellaneous sub-boxes in Box 7 for problems requiring domain-specific treatment), spanning all major structural types of NP-hard problems catalogued in Garey & Johnson (1979); (2) a three-tier reconnaissance system for landscape analysis (aerial survey, scout sampling, and mass sampling); and (3) a five-cut decision tree that maps landscape properties to optimal algorithm selection. Each sub-box is equipped with a formal objective function and a corresponding best-fit solver, with explicit literature support for each recommendation. The framework eliminates trial-and-error algorithm selection by providing a systematic pipeline: classify the problem, reconnoitre the landscape, classify the terrain, and deploy the matched algorithm. We demonstrate the framework's feasibility using Memetic Collapse-Chain Optimisation (MCCO) as a complete proof of concept for Box 6 (sequential decision problems), where landscape-guided deployment achieved a $3.56\times$ speedup with identical solution quality. The approach draws an explicit analogy to database normalisation: just as arbitrarily complex data can be decomposed into clean relational structures through a finite sequence of normal forms, arbitrarily complex NP problems can be classified into a finite set of landscape types through a progressive series of cuts. The framework does not claim to resolve $\mathrm{P} \stackrel{?}{=} \mathrm{NP}$; rather, it aims to minimise the practical gap between NP-hard problems and polynomial-time solvability by ensuring that every problem is attacked with the structurally optimal method.

Version 6 strengthens the framework along five dimensions: (i) best-fit solver recommendations for each sub-box are supported by literature citations (Martello & Toth 1990, Applegate et al. 2006, Nemhauser & Trotter 1975, Chvátal 1979, Dinur & Steurer 2014, and others); (ii) sub-box boundaries are refined so covering problems with different approximation structures are separated (Sub-box 2b is split into 2b-i for Vertex Cover/Set Cover and 2b-ii for Dominating Set); (iii) the universality claim is tightened—we state that the taxonomy spans all major structural types catalogued in Garey & Johnson (1979), and add a subsection classifying eight modern NP-hard problems (Neural Architecture Search, Quantum Circuit Compilation, Federated Learning selection, etc.) to demonstrate extensibility; (iv) a *Landscape-Method Compatibility Proposition* is introduced as the theoretical justification for classification: optimisation methods are effective only when their update operators align with the landscape's structural properties, implying that the (problem, method) space admits a natural compatibility partition that classification makes explicit; (v) three *universal quantitative indicators*—ruggedness R, constraint propagation density D_c, and gradient smoothness S_g—are defined, converting classification from expert judgement into reproducible measurement.

Version 7 consolidates the framework's formal and statistical foundations along six dimensions: (i) the *Landscape-Method Compatibility Proposition* is accompanied by a working definition of "structural morphism" (neighbourhood preservation and basin preservation) and an explicit *Status* remark acknowledging its standing as a structural principle with empirical support rather than a formally proven theorem; (ii) the *Adequacy Score* is redefined as an explicit piecewise step function of problem scale and difficulty rather than an unspecified $f(N \times D)$; (iii) *surrogate uncertainty propagation* is formalised via inverse-variance weighting, giving an explicit aggregation formula and standard-error expression for Gaussian-Process-backed difficulty scores; (iv) the multiple-testing correction section explicitly addresses score correlation, contrasting Bonferroni with Holm–Bonferroni and Benjamini–Hochberg alternatives for positive dependence; (v) the inapproximability citation for Dominating Set is corrected to the Feige (1998) Set-Cover reduction as primary source, with Dinur–Steurer (2014) retained as tightness reinforcement; (vi) the two separate speedup figures reported for Box 6 (MCCO's $24{,}000\times$ algorithmic speedup and the reconnaissance-guided $3.56\times$ deployment speedup) are now explicitly disambiguated to prevent baseline confusion. Version 7 also adds an honest limitation concerning the operational definition of the ruggedness indicator R: preliminary calibration experiments suggest that naive lag-1 autocorrelation has limited discriminating power across Box types, and candidate refinements are listed for future work.

Keywords: NP-hard, loss landscape, algorithm selection, fitness landscape analysis, combinatorial optimisation, meta-algorithm, landscape classification, reconnaissance, MCCO

摘要

本文提出一個基於損失曲面拓撲的 NP 難題統一分類框架。框架分三層運作：（1）七盒子問題分類法按「操作結構」組織（選擇、排序、分配、切割、著色、覆蓋、團、連接、邏輯滿足、路徑、路由、網路設計、排程、匹配、布局、序列決策），涵蓋 Garey & Johnson (1979) 目錄中所有主要結構類型；（2）三層偵察系統（空中偵查、斥候採樣、大量採樣）用於曲面分析；（3）五刀決策樹將曲面性質對應到最適演算法。各子盒子均配備正式目標函數與最適求解器，並附文獻支持。以模因坍縮鏈最佳化（MCCO）作為第六盒子的完整概念驗證，展示偵察導引部署實現 3.56 倍加速且解品質不變。

v6 強化框架五個面向：(i) 每個子盒子的最適求解器建議補上文獻引用；(ii) 子盒子邊界重新劃分（2b 拆為 2b-i 和 2b-ii）；(iii) 放棄「95%」大膽宣稱，改為「涵

蓋 Garey-Johnson 1979 所有主要結構類型」，並新增章節驗證框架對八個現代 NP 難題（NAS、量子電路編譯、聯邦學習等）的分類能力；(iv) 新增 Landscape-Method Compatibility 命題作為分類的理論正當性——最佳化方法只有在算子與 landscape 結構對齊時才有效，使（問題，方法）空間存在自然的相容性劃分；(v) 定義三個通用量化指標——粗糙度 R、約束傳播密度 D_c、梯度平滑度 S_g——將分類從專家判斷提升為可重現測量。

v7 在形式與統計基礎上強化六個面向：(i) Landscape-Method Compatibility 命題補上「結構性態射」的工作定義（保留鄰域關係與吸引盆）及明確的 *Status* remark，承認其為結構性原則而非已證明定理；(ii) Adequacy Score 改寫為問題規模與難度的明確分段階梯函數，取代先前未定義的 $f(N \times D)$；(iii) 代理模型不確定性傳播透過反變異數加權形式化，給出 GP 加權難度分數的明確公式與標準誤表達；(iv) Multiple testing correction 段落處理分數相關性，對比 Bonferroni 與 Holm–Bonferroni、Benjamini–Hochberg 替代方案；(v) Dominating Set 的不可近似性引用更正為 Feige (1998) 的 Set Cover reduction 為主引用，Dinur–Steurer (2014) 保留作為 tightness 補強；(vi) 盒子六中兩個加速倍數（MCCO 相對於 GA+MC 的 $24{,}000\times$ 核心加速與偵察引導的 $3.56\times$ 部署加速）明確消歧，避免基準混淆。v7 同時新增關於 ruggedness 指標 R 運算化定義的誠實 limitation：初步校準實驗顯示 naive lag-1 自相關在不同 Box 類別間區別力有限，並列出候選改進方向作為未來工作。

Contents

1 Introduction **8**
- 1.1 The Algorithm Selection Problem . . . 8
- 1.2 The Landscape Analogy . . . 8
- 1.3 The Database Normalisation Analogy . . . 8
- 1.4 Contributions . . . 8

2 The Seven-Box Taxonomy **9**
- 2.1 Classification Principle . . . 9
- 2.2 Why Classify? The Landscape-Method Compatibility Principle . . . 9
- 2.3 Box 1: Combinatorial Optimisation . . . 11
  - 2.3.1 Sub-box 1a: Selection (Choose a Subset) . . . 12
  - 2.3.2 Sub-box 1b: Ordering (Find Best Permutation) . . . 12
  - 2.3.3 Sub-box 1c: Allocation (Assign to Containers) . . . 12
  - 2.3.4 Sub-box 1d: Cutting (Determine Cut Positions) . . . 12
- 2.4 Box 2: Graph Theory . . . 13
  - 2.4.1 Sub-box 2a: Colouring (Partition Nodes Without Conflict) . . . 13
  - 2.4.2 Sub-box 2b-i: Covering with Constant-Factor Approximation . . . 13
  - 2.4.3 Sub-box 2b-ii: Covering without Constant-Factor Approximation . . . 13
  - 2.4.4 Sub-box 2c: Clique (Find Largest Fully Connected Subgraph) . . . 14
  - 2.4.5 Sub-box 2d: Connection (Minimum Cost to Connect Terminals) . . . 14
- 2.5 Box 3: Logical Satisfaction . . . 14
  - 2.5.1 Sub-box 3a: SAT (Satisfy All Clauses) . . . 14
  - 2.5.2 Sub-box 3b: CSP (Satisfy All Constraints) . . . 15
  - 2.5.3 Sub-box 3c: MAX-SAT (Satisfy As Many Clauses As Possible) . . . 15
- 2.6 Box 4: Path and Network . . . 15
  - 2.6.1 Sub-box 4a: Path (Visit All Nodes) . . . 15
  - 2.6.2 Sub-box 4b: Routing (Multi-Vehicle with Constraints) . . . 15
  - 2.6.3 Sub-box 4c: Network Design (Build Optimal Infrastructure) . . . 16
- 2.7 Box 5: Permutation and Assignment . . . 16
  - 2.7.1 Sub-box 5a: Scheduling (People to Time Slots) . . . 16
  - 2.7.2 Sub-box 5b: Matching (One-to-One Pairing) . . . 16
  - 2.7.3 Sub-box 5c: Layout (Objects to Locations) . . . 16
- 2.8 Box 6: Sequential Decision Under Uncertainty . . . 17
  - 2.8.1 Sub-box 6a: Financial Parameter Optimisation . . . 17
  - 2.8.2 Sub-box 6b: Medical Decision Sequences . . . 17
  - 2.8.3 Sub-box 6c: Control and Inventory . . . 17
- 2.9 Box 7: Miscellaneous / Unclassifiable . . . 17
  - 2.9.1 Sub-box 7a: Number-Theoretic . . . 17
  - 2.9.2 Sub-box 7b: Computational Geometry . . . 17
  - 2.9.3 Sub-box 7c: Hybrid / Cross-Box . . . 18
- 2.10 Classification of Modern NP-Hard Problems . . . 18

3 The Three-Tier Reconnaissance System **18**
- 3.1 Tier 1: Aerial Survey (Zero Cost) . . . 19
  - 3.1.1 Three Universal Quantitative Indicators . . . 19
- 3.2 Tier 2: Scout Sampling (Low Cost) . . . 20
- 3.3 Tier 3: Mass Sampling (Medium Cost) . . . 20

3.4 Reconnaissance Coverage . . . 21
3.5 Statistical Foundations of Reconnaissance . . . 21
3.5.1 Type I and Type II Errors in Reconnaissance . . . 21
3.5.2 Power Analysis for Scout Count . . . 22
3.5.3 Multiple Testing Correction . . . 22
3.5.4 Effect Size Estimation . . . 23
3.5.5 Surrogate Uncertainty Propagation . . . 24

4 The Five-Cut Decision Tree 24

5 Landscape Types and Algorithm Matching 25

6 Scout-Based Algorithm Adjustment 26
6.1 Box 1: Combinatorial Optimisation . . . 26
6.2 Box 2: Graph Theory . . . 26
6.3 Box 3: Logical Satisfaction . . . 26
6.4 Box 4: Path and Network . . . 27
6.5 Box 5: Permutation and Assignment . . . 27
6.6 Box 6: Sequential Decision . . . 27
6.7 Key Observation . . . 27

7 Landscape-Based Parameter Configuration 27
7.1 Genetic Algorithm Family . . . 28
7.2 Simulated Annealing . . . 28
7.3 Local Search / Tabu Search . . . 28
7.4 Gradient-Based / BPTT . . . 28
7.5 Branch and Bound . . . 28
7.6 SAT Solvers . . . 28
7.7 Sub-Box Specific Configurations . . . 29

8 Landscape Transformation 30
8.1 Transformation Methods . . . 30
8.2 Box-Specific Transformation Applicability . . . 30
8.3 Difficulty Scoring . . . 31
8.4 The Reconnaissance–Transformation Loop . . . 32
8.5 Four-Layer Stopping Conditions . . . 32
8.6 Transformation as Computational Investment . . . 33
8.7 Precision Recovery . . . 33
8.8 Updated Complete Pipeline . . . 34

9 Landscape Cutting (Decomposition) 34
9.1 Cutting Principle . . . 34
9.2 Cutting Tools . . . 35
9.3 Box-Specific Cuttability . . . 35
9.4 Transform First, Then Cut . . . 35
9.5 Parallel Deployment . . . 35
9.6 Five Cutting Considerations . . . 36

10 The Evaluation Matrix **36**
10.1 Matrix Construction . . . 36
10.2 Cost . . . 37
10.3 Empirical Validation . . . 37
10.4 Cutting Quality Check . . . 37
10.5 Score Significance Testing . . . 38

11 Algorithm Adequacy Scoring **39**
11.1 Adequacy Score . . . 39

12 Proof of Concept: Box 6 and MCCO **40**
12.1 Landscape-Guided Deployment Result . . . 40
12.2 Inverse-Variance Weighting Result . . . 41
12.3 Implications for Other Boxes . . . 41

13 Worked Example: TSP with Framework vs Blind Deployment **41**
13.1 Problem Instance . . . 41
13.2 Framework Execution (Ten-Step Pipeline) . . . 41
13.3 Three-Way Comparison . . . 42
13.4 Key Observations . . . 42
13.5 Scaling Test: 20 and 50 Cities . . . 42
13.6 How Close to P? . . . 43
13.7 TSPLIB Benchmark Validation . . . 44
13.8 Multidimensional Knapsack Benchmark (Sub-box 1a) . . . 44
13.9 Graph Coloring Benchmark (Sub-box 2a) . . . 45
13.10 3-SAT Benchmark (Sub-box 3a) . . . 46
13.11 Vehicle Routing Benchmark (Sub-box 4b) . . . 47
13.12 Parallel Machine Scheduling Benchmark (Sub-box 5a) . . . 48
13.13 Improvement Scales with Problem Size . . . 49
13.14 Scalability Validation (1,000+ Instances) . . . 49

14 Complexity Reference **50**

15 Known Challenges and Preventive Measures **50**

16 Discussion **51**
16.1 Relation to Existing Work . . . 51
16.1.1 Classical Problem Taxonomy . . . 51
16.1.2 Fitness Landscape Analysis (FLA) and Exploratory Landscape Analysis (ELA) . . . 51
16.1.3 Algorithm Selection Problem . . . 51
16.1.4 No Free Lunch Theorem . . . 52
16.1.5 Parameterized Complexity . . . 52
16.1.6 Learning-Augmented Algorithms . . . 52
16.1.7 Summary . . . 53
16.2 Limitations . . . 53
16.3 Not P = NP, But Practical Convergence . . . 54
16.4 An AI-Executable Protocol . . . 55

17 Conclusion **56**

1 Introduction

1.1 The Algorithm Selection Problem

When faced with an NP-hard problem, practitioners typically proceed by trial and error: they implement several candidate algorithms, run benchmarks, and select whichever performs best on their particular instance. This process is time-consuming, wasteful, and provides no guarantee that the selected algorithm is structurally suited to the problem's landscape. It is analogous to attempting to climb a mountain without first looking at it—choosing equipment based on habit rather than terrain.

本文提出一個根本性的替代方案：不要盲目選擇演算法，先看清楚問題的「山」長什麼樣。

1.2 The Landscape Analogy

Every optimisation problem defines a *loss landscape*: a mapping from the solution space to objective values. The topology of this landscape—whether it is smooth or rugged, unimodal or multimodal, continuous or discrete—determines which algorithms will be effective and which will waste computational resources. A gradient-based method on a rugged discrete landscape is like wearing ice crampons on a gravel slope; a genetic algorithm on a smooth unimodal surface is like deploying an army to climb a hill.

The central thesis of this paper is:

> *The computational cost of solving an NP-hard problem is a function of its loss landscape topology, not a fixed property of the problem class. By classifying the landscape before selecting the algorithm, the practical gap between NP and P can be substantially reduced.*
>
> NP 難題的計算成本是其損失曲面拓撲的函數，而非問題類別的固定屬性。在選擇演算法之前先分類曲面，可以大幅縮小 NP 與 P 之間的實務差距。

1.3 The Database Normalisation Analogy

Database normalisation theory established that arbitrarily complex relational data can be decomposed into clean structures through a finite sequence of normal forms (1NF $\rightarrow$ 2NF $\rightarrow \cdots \rightarrow$ 5NF). Not all datasets require full normalisation—some are clean at 2NF, others need 4NF—but the framework guarantees that a finite number of steps suffices for any dataset.

We propose an analogous framework for NP-hard problems: a finite sequence of classification "cuts" that progressively refines the landscape characterisation until the optimal algorithm is determined. Some problems are classified in two cuts; others require five. But the framework guarantees completeness: every NP-hard problem can be classified.

1.4 Contributions

This paper makes six contributions:

1. Defines a **seven-box problem taxonomy** organised by operational structure, spanning all major structural types of NP-hard problems catalogued in Garey & Johnson (1979), with a residual seventh box for problems requiring domain-specific treatment.

2. Introduces a **three-tier reconnaissance system** (aerial survey, scout sampling, mass sampling) for landscape analysis, with box-specific indicators for each tier.

3. Proposes a **five-cut decision tree** that maps landscape properties to optimal algorithm families.

4. Provides **formal objective functions** for all 24 sub-boxes, enabling immediate mathematical characterisation of any classified problem.

5. Presents **MCCO** (Memetic Collapse-Chain Optimisation) as a complete proof of concept for Box 6, demonstrating that landscape-guided deployment achieves $3.56\times$ speedup with identical solution quality [tonyrao, 2026a].

6. Identifies **potential unification bases** for each box—mathematical substrates analogous to the collapse chain that could enable structural acceleration within each problem class.

2 The Seven-Box Taxonomy

2.1 Classification Principle

We classify NP-hard problems not by their application domain (logistics, biology, finance) but by their *structural type*—the mathematical form of the decision variables, constraints, and objective function. This ensures that problems with identical landscape properties are grouped together regardless of origin.

Sub-box boundaries are determined by landscape structure and approximation hardness, not by superficial semantic similarity. Two problems may share the same decision vocabulary (both select a subset of vertices, for instance) yet occupy fundamentally different sub-boxes if their LP integrality gaps, approximation bounds, or local landscape topologies differ. Conversely, two problems from different application domains (e.g., Set Cover from combinatorial optimisation and graph-based covering in network design) belong to the same sub-box if their mathematical structure is equivalent.

The canonical example is the distinction between Sub-box 2b-i (Vertex Cover, Set Cover, which admit constant-factor or logarithmic approximations via LP rounding) and Sub-box 2b-ii (Dominating Set, which does not admit any constant-factor approximation). Both sub-boxes involve selecting a vertex subset under covering-type constraints, but the underlying landscape differs enough to warrant separate algorithmic treatment.

問題的分類依據不是應用領域，而是結構類型——決策變數、約束和目標函數的數學形式。子類別的邊界由 landscape 結構和近似難度決定，而非表面的語義相似性。

2.2 Why Classify? The Landscape-Method Compatibility Principle

Before presenting the taxonomy itself, we address a foundational question: why is classification necessary at all? Could one simply apply a universal optimiser (e.g., large-population genetic algorithm, deep reinforcement learning) to every NP-hard problem?

The No Free Lunch theorem [Wolpert & Macready, 1997] provides the starting answer: no single optimiser is best on all problems. But NFL alone does not justify a classification framework—it only says that *some* problem-dependent selection is needed. We propose a stronger claim, which motivates the taxonomy.

Proposition 1 (Landscape-Method Compatibility)**.** *An optimisation method is effective on an NP-hard problem only if the method's update operators align with the structural properties of the underlying loss landscape. Formally, let $\mathcal{L}$ denote the landscape structure of a problem instance (its neighbourhood graph, basin topology, constraint geometry), and let $\mathcal{O}$ denote the set of operators defining a method (move generation, selection, acceptance, termination). The method achieves polynomial-effort approximation on the instance only if there exists a structural morphism from $\mathcal{O}$ to $\mathcal{L}$ preserving neighbourhood relations and attraction basins.*

Remark 1 (Status of the proposition)**.** *Proposition 1 is stated as a structural principle rather than a formally proven theorem. A rigorous proof would require a formal definition of "structural morphism" between operator sets and landscape structures—in particular, a category-theoretic or topological framework in which neighbourhood-preserving maps between $\mathcal{O}$ and $\mathcal{L}$ can be rigorously compared—which is beyond the scope of this paper.*

Working definition. By structural morphism *we mean, informally, a map $\varphi : \mathcal{O} \to \mathcal{L}$ such that: (i) φ preserves the neighbourhood relation—if y is reachable from x by a single application of an operator in $\mathcal{O}$, then $\varphi(y)$ is landscape-adjacent to $\varphi(x)$ in the neighbourhood graph of $\mathcal{L}$; (ii) φ preserves attraction basins—the convergence dynamics induced by $\mathcal{O}$ on the image of φ respect the basin boundaries defined by $\mathcal{L}$. Gradient descent is structurally compatible with smooth continuous landscapes because its update operator (move in the direction of $-\nabla f$) preserves both properties with respect to the Euclidean neighbourhood and basin structure of such landscapes. It is structurally incompatible with discrete constraint landscapes because no such φ exists—there is no differentiable surface on which the gradient operator can act. A formal category-theoretic treatment of this map is left to future work.*

We offer instead a three-part structural argument: the canonical mismatches enumerated in Remark 4 demonstrate, on three well-documented examples, that structural misalignment between operators and landscape produces super-polynomial performance degradation that no amount of parameter tuning can remedy. We treat this as strong empirical support for the proposition and leave a formal proof to future work.

本命題以結構性原則形式陳述，而非形式化證明的定理。嚴格證明需要先為「結構性態射」建立形式化定義——特別是能夠嚴格比較 $\mathcal{O}$ 與 $\mathcal{L}$ 之間保留鄰域結構的映射的範疇論或拓撲框架——此超出本文範圍。

工作定義。「結構性態射」非形式化地定義為一個映射 $\varphi : \mathcal{O} \to \mathcal{L}$，滿足：(i) 保留鄰域關係——若 y 可由 $\mathcal{O}$ 中的單次算子操作從 x 到達，則 $\varphi(y)$ 在 $\mathcal{L}$ 的鄰域圖中與 $\varphi(x)$ 相鄰；(ii) 保留吸引盆——$\mathcal{O}$ 在 φ 像上誘導的收斂動力學尊重 $\mathcal{L}$ 所定義的盆邊界。梯度下降與平滑連續曲面結構相容，因其更新算子（沿 $-\nabla f$ 方向移動）相對於此類曲面的歐幾里得鄰域與盆結構保留兩性質；其與離散約束曲面結構不相容，因為不存在此種 φ——沒有可作用梯度算子的可微曲面。此映射的形式化範疇論處理留作未來工作。

我們改以三部分結構性論證作為支撐：Remark 4 列舉的三個典型不匹配案例展示，算子與 landscape 之間的結構不一致會導致超多項式的效能退化，無法透過參數調整彌補。我們以此作為本命題的強經驗支持，並將形式化證明留作未來工作。

Remark 2 (Interpretation)**.** *The proposition does not state that structurally matched methods are guaranteed to succeed (the problem may still be computationally hard even with perfect matching). It states the converse: structurally mismatched methods cannot succeed in polynomial effort, regardless of tuning. A gradient-based optimiser cannot solve a SAT instance not because gradient descent is weak, but because SAT's discrete constraint landscape has no differentiable surface for the gradient to descend. Conversely, CDCL cannot solve a continuous optimisation problem because CDCL's clause-learning operator has no meaning in continuous space.*

Remark 3 (Implication for classification)**.** *Proposition 1 implies that the space of (problem, method) pairs admits a natural partition: pairs that are structurally compatible, and pairs that are not. A classification framework identifies these compatibility classes and provides a mapping from problem structure to compatible methods. Without such a framework, practitioners must discover compatibility empirically—often at the cost of weeks or months of trial and error. With such a framework, compatibility is determined by structural inspection in bounded time.*

Remark 4 (Three canonical mismatches)**.** *The proposition is illustrated by three well-documented failure modes in the literature.*

Mismatch 1: Gradient descent on 3-SAT at the phase transition. *The landscape of 3-SAT near* $\alpha \approx 4.26$ *has no differentiable structure—the objective (number of satisfied clauses) changes discretely with each variable flip. Gradient-based methods have no gradient to follow. Attempts to apply continuous relaxations (e.g., semidefinite programming) succeed for small instances but degrade rapidly with* N*, while CDCL succeeds on instances orders of magnitude larger [Moskewicz et al., 2001, Een & Sörensson, 2003]. The mismatch is not a matter of tuning; it is structural.*

Mismatch 2: CDCL on continuous optimisation. *CDCL's clause-learning operator has no meaning in continuous space—there are no clauses to learn from, and no unit propagation to perform. Applying CDCL to a nonlinear programming problem produces no useful output. Conversely, applying nonlinear solvers to SAT requires discretising the continuous solution, which discards the solver's entire advantage.*

Mismatch 3: Generic GA on Dominating Set without decomposition. *Genetic algorithms with bitstring representation and uniform crossover treat Dominating Set as a generic selection problem. However, the domination constraint is global—flipping one bit can invalidate coverage at arbitrary distant nodes—violating the locality assumption underlying GA's crossover and mutation operators. Empirically, GA without problem-specific operators performs worse than the greedy* $\ln n$*-approximation on all but the smallest instances [Cygan et al., 2015].*

In each mismatch, the failure is not remediable by parameter tuning. The operators and the landscape are structurally incompatible. Proposition 1 codifies this empirical pattern: matched operators are necessary (though not sufficient) for polynomial-effort approximation.

The five-cut decision tree (Section 4) is the operational realisation of Proposition 1: it transforms the taxonomy from a descriptive grouping into a *decision rule* of the form

$$\text{if landscape type} = X \text{ then use method} \in \mathcal{M}_X,$$

where $\mathcal{M}_X$ is the set of methods structurally compatible with landscape type X. This elevates the framework from classification to actionable guidance.

分類為何必要？No Free Lunch 定理僅說明需要問題相關的選擇，但未指定如何選擇。我們提出 Landscape-Method Compatibility Principle：最佳化方法只有在其更新算子與底層 landscape 的結構性質對齊時才能有效。此命題意味著（問題，方法）配對空間存在自然劃分——結構相容者與不相容者。分類框架識別這些相容性類別，將分類從「描述」提升為「if-then 決策規則」。

2.3 Box 1: Combinatorial Optimisation

Problems involving selection, ordering, allocation, or cutting of discrete objects to optimise an objective under constraints.

2.3.1 Sub-box 1a: Selection (Choose a Subset)

Select a subset from N items to maximise/minimise an objective under capacity or cardinality constraints.

Canonical problem: 0/1 Knapsack.

$$\max \sum_{i=1}^{N} v_i x_i \quad \text{s.t.} \quad \sum_{i=1}^{N} w_i x_i \leq W, \quad x_i \in \{0,1\} \tag{1}$$

Other members: Maximum Independent Set, Feature Selection, Capital Budgeting.

Landscape: Constrained boundary, LP relaxation yields smooth continuous approximation.

Best-fit solvers: Dynamic Programming (small N) [Martello & Toth, 1990], Greedy + LP relaxation (large N) [Pisinger, 2005], Branch and Bound (exact) [Martello & Toth, 1990].

2.3.2 Sub-box 1b: Ordering (Find Best Permutation)

Find the optimal ordering of N elements.

Canonical problem: Travelling Salesman Problem (TSP).

$$\min \sum_{i=1}^{N} d(\pi_i, \pi_{i+1}) + d(\pi_N, \pi_1) \tag{2}$$

Other members: Flow Shop Scheduling, Single Machine Scheduling.

Landscape: Multimodal with structured basins; swap neighbourhood defines adjacency.

Best-fit solvers: 2-opt / 3-opt local search [Applegate et al., 2006], GA + local search (memetic) [Helsgaun, 2000], Ant Colony Optimisation [Dorigo & Gambardella, 1997].

2.3.3 Sub-box 1c: Allocation (Assign to Containers)

Distribute N items into M containers under capacity constraints.

Canonical problem: Bin Packing.

$$\min \sum_{j=1}^{M} y_j \quad \text{s.t.} \quad \sum_{i \in B_j} s_i \leq C \; \forall j, \quad y_j \in \{0,1\} \tag{3}$$

Other members: Parallel Machine Scheduling, Resource Allocation.

Landscape: Flat plateau with narrow valleys; many solutions of similar quality.

Best-fit solvers: First Fit Decreasing (greedy) [Coffman et al., 2013], Simulated Annealing [Kirkpatrick et al., 1983], Constraint Programming.

2.3.4 Sub-box 1d: Cutting (Determine Cut Positions)

Determine optimal cutting positions on continuous or discrete material.

Canonical problem: Cutting Stock.

$$\min \sum_{j=1}^{M} y_j \quad \text{s.t.} \quad \sum_{j} a_{ij} y_j \geq d_i \; \forall i \tag{4}$$

Other members: 2D Nesting, Strip Packing.

Landscape: Mixed continuous/discrete; cutting positions are continuous but patterns are discrete.

Best-fit solvers: Column Generation [Gilmore & Gomory, 1961], Greedy + local search.

2.4 Box 2: Graph Theory

Problems defined on graph structures $G = (V, E)$ where decisions involve colouring, covering, or connecting nodes.

2.4.1 Sub-box 2a: Colouring (Partition Nodes Without Conflict)

Canonical problem: Graph Colouring.

$$\min k \quad \text{s.t.} \quad c(u) \neq c(v)\ \forall (u, v) \in E \tag{5}$$

Other members: Frequency Assignment, Exam Scheduling.

Landscape: Rugged with constraint cliffs.

Best-fit solvers: Greedy colouring (DSATUR) [Brélaz, 1979], Tabu Search [Galinier & Hertz, 2006, Glover, 1989], Constraint Programming.

2.4.2 Sub-box 2b-i: Covering with Constant-Factor Approximation

Problems where a constant-factor approximation algorithm is known and the LP relaxation provides tight bounds.

Canonical problem: Vertex Cover.

$$\min |S| \quad \text{s.t.} \quad \forall (u, v) \in E,\ u \in S \text{ or } v \in S \tag{6}$$

Other members: Set Cover (all variants, with logarithmic approximation), Edge Cover.

Landscape: Relatively smooth; LP relaxation yields half-integral solutions for Vertex Cover (Nemhauser-Trotter); greedy provides $O(\log n)$ approximation for Set Cover.

Best-fit solvers: Greedy (2-approximation for Vertex Cover [Garey & Johnson, 1979], $\ln n$-approximation for Set Cover [Chvátal, 1979]), LP relaxation with rounding [Nemhauser & Trotter, 1975], Branch and Bound.

2.4.3 Sub-box 2b-ii: Covering without Constant-Factor Approximation

Problems where no constant-factor approximation is known (under standard complexity assumptions) and LP relaxation has large integrality gap.

Canonical problem: Dominating Set.

$$\min |S| \quad \text{s.t.} \quad \forall v \in V,\ v \in S \text{ or } \exists u \in S \text{ with } (u, v) \in E \tag{7}$$

Other members: Independent Dominating Set, Connected Dominating Set, Total Dominating Set.

Landscape: Rugged with many deceptive local optima; LP integrality gap is $\Theta(\log n)$; approximating within $(1 - \epsilon) \ln n$ is NP-hard, via a reduction from Set Cover [Feige, 1998] whose tight inapproximability has been reinforced by subsequent parallel repetition results [Dinur & Steurer, 2014].

Best-fit solvers: Greedy (logarithmic approximation) [Chvátal, 1979], ILP for small instances, parameterised algorithms (fixed-parameter tractable on specific graph classes) [Cygan et al., 2015, Downey & Fellows, 2013].

Distinction from 2b-i: Although Vertex Cover and Dominating Set both involve selecting a subset of vertices, their computational structure differs fundamentally. Vertex Cover admits a

2-approximation via LP rounding; Dominating Set does not admit any constant-factor approximation unless P=NP. This difference reflects the landscape: the covering constraint in Vertex Cover has local structure (each edge must be covered by one of two endpoints), while the domination constraint has global structure (each vertex must be in the closed neighbourhood of the cover set).

2.4.4 Sub-box 2c: Clique (Find Largest Fully Connected Subgraph)

Canonical problem: Maximum Clique.

$$\max |C| \quad \text{s.t.} \quad \forall u, v \in C,\ (u, v) \in E \tag{8}$$

Other members: Maximum Independent Set (complement graph).
Landscape: Deceptive; locally promising regions often lead to suboptimal solutions.
Best-fit solvers: Branch and Bound [Bomze et al., 1999], large-population GA [Holland, 1975].

2.4.5 Sub-box 2d: Connection (Minimum Cost to Connect Terminals)

Canonical problem: Steiner Tree.

$$\min \sum_{e \in T} w_e \quad \text{s.t.} \quad T \text{ connects all terminal nodes} \tag{9}$$

Other members: Minimum Spanning Tree variants, Network Design.
Landscape: Multimodal but with relatively small solution space.
Best-fit solvers: Approximation algorithms [Hwang & Richards, 1992], DP (small scale).

2.5 Box 3: Logical Satisfaction

Problems involving Boolean variables and logical constraints.

2.5.1 Sub-box 3a: SAT (Satisfy All Clauses)

Canonical problem: 3-SAT.

$$\text{find } \mathbf{x} \in \{0, 1\}^N \text{ s.t. } \bigwedge_{j=1}^{M} C_j(\mathbf{x}) = \text{true} \tag{10}$$

Note: 2-SAT is in P (linear time). 3-SAT exhibits a phase transition at clause-to-variable ratio $\alpha \approx 4.26$: instances below this threshold are easy; instances near it are extremely hard.
Landscape: Smooth at low density; extremely rugged near phase transition.
Best-fit solvers: DPLL [Davis et al., 1962], CDCL (Conflict-Driven Clause Learning) [Marques-Silva & Sakallah, 1999, Moskewicz et al., 2001], WalkSAT [Selman et al., 1994].

2.5.2 Sub-box 3b: CSP (Satisfy All Constraints)

Canonical problem: Constraint Satisfaction Problem.

$$\text{find } \mathbf{x} \in D_1 \times \cdots \times D_N \text{ s.t. all constraints } R_k(\mathbf{x}) \text{ satisfied} \tag{11}$$

Other members: Sudoku, Map Colouring, Scheduling Constraints.
Best-fit solvers: Backtracking + Arc Consistency propagation, Constraint Programming [Rossi et al., 2006].

2.5.3 Sub-box 3c: MAX-SAT (Satisfy As Many Clauses As Possible)

Canonical problem: Maximum Satisfiability.

$$\max \sum_{j=1}^{M} C_j(\mathbf{x}), \quad \mathbf{x} \in \{0,1\}^N \tag{12}$$

Landscape: Similar to SAT but with a continuous objective (count of satisfied clauses).
Best-fit solvers: Branch and Bound, Simulated Annealing [Kirkpatrick et al., 1983], GA [Holland, 1975].

2.6 Box 4: Path and Network

Problems involving optimal routing, paths, or network construction.

2.6.1 Sub-box 4a: Path (Visit All Nodes)

Canonical problem: Hamiltonian Path.

$$\text{find path visiting all } v \in V \text{ exactly once}, \quad \min \sum_{e \in P} w_e \tag{13}$$

Landscape: Similar to TSP; multimodal with swap neighbourhood.
Best-fit solvers: GA + local search [Holland, 1975], Branch and Bound [Ahuja et al., 1993].

2.6.2 Sub-box 4b: Routing (Multi-Vehicle with Constraints)

Canonical problem: Vehicle Routing Problem (VRP).

$$\min \sum_{k=1}^{K} \sum_{e \in R_k} d_e \quad \text{s.t.} \quad \sum_{i \in R_k} q_i \leq Q \; \forall k, \quad \text{time windows} \tag{14}$$

Other members: Pickup-Delivery, VRP with Time Windows.
Landscape: Multimodal with constraint cliffs (capacity, time windows).
Best-fit solvers: Clarke-Wright (greedy) [Toth & Vigo, 2014], Large Neighbourhood Search (LNS) [Shaw, 1998], Ant Colony [Dorigo & Gambardella, 1997].

2.6.3 Sub-box 4c: Network Design (Build Optimal Infrastructure)

Canonical problem: Facility Location.

$$\min \sum_j f_j y_j + \sum_i \sum_j c_{ij} x_{ij} \quad \text{s.t.} \quad \sum_j x_{ij} = 1 \; \forall i, \quad x_{ij} \leq y_j \tag{15}$$

Other members: Steiner Network, Hub Location.
Best-fit solvers: LP relaxation + Branch and Bound [Ahuja et al., 1993], Lagrangian relaxation, Greedy.

2.7 Box 5: Permutation and Assignment

Problems involving assigning entities to slots under complex constraints.

2.7.1 Sub-box 5a: Scheduling (People to Time Slots)

Canonical problem: Nurse Scheduling.

$$\min \sum_{i,t} \text{penalty}(i,t) \quad \text{s.t. coverage, fairness, legal constraints} \tag{16}$$

Other members: Airline Crew Scheduling, University Timetabling.
Landscape: Constraint cliffs dominate; feasible region is narrow.
Best-fit solvers: Constraint Programming [Rossi et al., 2006], Tabu Search [Brandimarte, 1993, Glover, 1989], Integer Programming [Pinedo, 2016].

2.7.2 Sub-box 5b: Matching (One-to-One Pairing)

Canonical problem: Weighted Bipartite Matching.

$$\max \sum_{(i,j) \in M} w_{ij} \quad \text{s.t. each node matched at most once} \tag{17}$$

Note: Unweighted bipartite matching is in P (Hungarian Algorithm).
Best-fit solvers: Hungarian Algorithm (P for bipartite) [Kuhn, 1955], Auction Algorithm [Burkard et al., 2009].

2.7.3 Sub-box 5c: Layout (Objects to Locations)

Canonical problem: Quadratic Assignment Problem (QAP).

$$\min \sum_i \sum_j f_{ij} \cdot d_{\pi(i)\pi(j)}, \quad \pi \in S_N \tag{18}$$

Other members: VLSI Layout, Facility Layout.
Landscape: Extremely rugged; among the hardest combinatorial problems.
Best-fit solvers: Simulated Annealing [Kirkpatrick et al., 1983, Loiola et al., 2007], large-population GA [Loiola et al., 2007].

2.8 Box 6: Sequential Decision Under Uncertainty

Problems involving sequential observations, dynamic state updates, and decisions under ontological indeterminacy.

2.8.1 Sub-box 6a: Financial Parameter Optimisation

Canonical problem: Collapse-chain parameter optimisation.

$$\min_{\theta} \mathbb{E}_{\tau \sim \text{MC}} \left[\sum_{t} w_t \cdot L(\theta, \gamma_t, \tau_t) \right] \tag{19}$$

Landscape: Continuous, differentiable, analytic Jacobians, Chain Forgetting guarantees gradient stability.

Best-fit solver: MCCO [tonyrao, 2026a] — 24,000× speedup over GA+MC demonstrated.

2.8.2 Sub-box 6b: Medical Decision Sequences

$$\min_{\pi,\theta} \mathbb{E}\left[\text{harm}(\pi, \theta) + \text{cost}(\pi)\right] \quad \text{s.t.} \quad \gamma_t = \Phi(O_t, \gamma_{t-1}) \tag{20}$$

Landscape: Dynamic collapse; mixed continuous (dosage) and discrete (treatment order).

Best-fit solver: MCCO variant (GA for discrete, BPTT for continuous).

2.8.3 Sub-box 6c: Control and Inventory

$$\min_{\mathbf{u}} \mathbb{E}\left[\sum_{t} c(x_t, u_t) \right] \quad \text{s.t.} \quad x_{t+1} = f(x_t, u_t, w_t) \tag{21}$$

Landscape: Continuous, dynamic, depends on whether analytic Jacobians exist.

Best-fit solver: MCCO (if Jacobians available), Reinforcement Learning (if not).

2.9 Box 7: Miscellaneous / Unclassifiable

Problems that do not fit cleanly into Boxes 1–6 due to unique mathematical structure.

2.9.1 Sub-box 7a: Number-Theoretic

Canonical problem: Integer Factorisation.

$$\text{find } p, q \text{ s.t. } N = p \times q \tag{22}$$

Treatment: Domain-specific methods (Number Field Sieve, Shor's Algorithm). Not amenable to general landscape analysis.

2.9.2 Sub-box 7b: Computational Geometry

Specialised geometric problems (e.g., minimum enclosing variants). Dedicated geometric algorithms apply.

2.9.3 Sub-box 7c: Hybrid / Cross-Box

Problems simultaneously involving multiple boxes (e.g., routing + scheduling + assignment). Decompose into sub-problems, classify each sub-problem into its respective box, and solve separately or via coordination.

2.10 Classification of Modern NP-Hard Problems

The Garey & Johnson (1979) catalogue predates many NP-hard problems that have emerged in modern computing. A natural question is whether the taxonomy extends to these newer problems or whether modern problems require entirely new categories. We argue that the taxonomy extends naturally, because classification is based on operational structure rather than application semantics, and modern NP-hard problems tend to inherit their structural difficulty from classical operational types.

Table 1: Classification of representative modern NP-hard problems.

Problem	**Assigned Sub-box**	**Structural justification**
Neural Architecture Search	5c (Layout)	Assigning operations (conv, pool, attention) to network positions under structural constraints; objective is non-linear in position.
Hyperparameter Optimisation	1a (Selection) or 5a (Scheduling)	Select a subset of configurations (1a), or schedule configurations across resources under budget (5a); depending on formulation.
Graph Neural Network subgraph matching	2c (Clique)	Find largest connected common substructure.
Federated Learning client selection	1a (Selection)	Select clients under a bandwidth/fairness budget to maximise global model quality.
Quantum Circuit Compilation	5c (Layout) + 2b-i (Covering)	Assign logical qubits to physical qubits (5c); cover two-qubit gates with available connectivity (2b-i).
Blockchain Shard Assignment	5c (Layout)	Assign transactions to shards to minimise cross-shard communication.
Data-centre VM Placement	1c (Allocation)	Pack VMs into physical hosts under capacity constraints (Bin-packing variant).
Model Merging / Parameter Pruning	1a (Selection)	Select a subset of parameters to retain under a size budget.

None of these modern problems required creating a new sub-box. All fit into existing categories by their operational structure. This supports the claim that the taxonomy's categories are structurally fundamental rather than historically accidental.

現代 NP 難題分類驗證：神經架構搜尋、超參數最佳化、GNN 子圖匹配、聯邦學習選擇、量子電路編譯、區塊鏈分片、VM 放置、模型合併等現代問題均可自然納入既有七盒子，無需新增類別。這支持「分類類別是結構性的，而非歷史偶然」的主張。

3 The Three-Tier Reconnaissance System

Before selecting an algorithm, the landscape must be observed. We propose three tiers of reconnaissance, applied in order of increasing cost.

3.1 Tier 1: Aerial Survey (Zero Cost)

Analyse the problem instance's structural statistics without evaluating any solution. Each box has its own key indicators:

Box	Aerial Indicator	What It Reveals
1 (Combinatorial)	Item weight/value distribution	Constraint tightness
2 (Graph)	Degree distribution, density	Ruggedness
3 (Logic)	Clause/variable ratio α	Phase transition proximity
4 (Path/Network)	Spatial clustering of nodes	Number of basins
5 (Permutation)	Hard constraint ratio	Feasible region width
6 (Sequential)	κ^*, Hurst H, ρ	Learnability, stability

空中偵查不需要計算任何解的目標值，僅分析問題實例的結構統計指標。每個盒子有各自的關鍵指標。成本為零。

3.1.1 Three Universal Quantitative Indicators

The box-specific indicators in the table above are tuned to each problem family, but three *universal* quantitative indicators apply across all boxes. These provide the formal criterion that grounds the classification in measurable structure rather than semantic grouping.

(I) Ruggedness R. The autocorrelation of the objective value along a random walk on the neighbourhood graph:

$$R = 1 - \rho_1, \quad \rho_1 = \text{Corr}(f(x_t), f(x_{t+1}))$$

where x_{t+1} is a neighbour of x_t (one-step move in the problem-specific neighbourhood). $R \approx 0$ indicates a smooth landscape; $R \approx 1$ indicates an uncorrelated, rugged landscape. Computable from $O(L)$ neighbour evaluations (L a short walk length, typically $L = 100$–1000).

(II) Constraint Propagation Density D_c. The average fraction of variables whose domain is reduced by unit propagation after fixing one variable:

$$D_c = \frac{1}{N} \sum_{i=1}^{N} \frac{|\text{propagated}(x_i)|}{N-1}$$

where propagated(x_i) is the set of variables whose domain shrinks when x_i is fixed. D_c close to 0 indicates weak propagation (no global structure to exploit); D_c close to 1 indicates strong propagation (constraint reasoning is effective). Computable in $O(N^2)$ time via a single propagation sweep per variable.

(III) Gradient Smoothness S_g. For problems admitting a continuous relaxation, the Lipschitz constant of the gradient of the LP/convex relaxation:

$$S_g = \max_{x,y \in \mathcal{X}_{\text{rel}}} \frac{\|\nabla f_{\text{rel}}(x) - \nabla f_{\text{rel}}(y)\|}{\|x - y\|}$$

where f_{rel} is the relaxation and $\mathcal{X}_{\text{rel}}$ the relaxed domain. Small S_g indicates that gradient-based methods can be applied to the relaxation and rounded; large S_g indicates rapid gradient fluctuations that disqualify gradient methods. For discrete problems with no meaningful relaxation, $S_g = \infty$ by convention.

Table 2: Expected ranges of (R, D_c, S_g) for each Box, grounding classification in measurable quantities.

Box	R	D_c	S_g	**Landscape character**
1 (Combinatorial)	low–mid	low	small–mid	LP-relaxable, moderately rugged
2a (Colouring)	high	high	∞	rugged, constraint-dominated
2b-i (Covering)	low	mid–high	small	smooth, LP-tight
2b-ii (Dominating)	mid–high	mid–high	mid	rugged with constraint structure
3 (Logic)	high	very high	∞	discrete, constraint-dominated
4 (Path/Network)	mid	low	small–mid	spatial, relaxable
5 (Permutation)	high	high	∞	feasibility-dominated
6 (Sequential)	time-varying	n/a	n/a	depends on regime (see κ^*)

Classification as measurement. The three indicators (R, D_c, S_g) form a three-dimensional feature space in which each Box occupies a region. Given a novel problem, one computes (R, D_c, S_g) and assigns the problem to the Box whose region it falls into. This converts the classification from expert judgement into reproducible measurement. Two researchers computing the same indicators on the same instance will assign it to the same Box.

三個通用量化指標：粗糙度 R（相鄰解目標值自相關）、約束傳播密度 D_c（固定變數後其他變數域縮減比例）、梯度平滑度 S_g（連續鬆弛的梯度 Lipschitz 常數）。三個指標形成三維特徵空間，每個盒子佔據特定區域。分類從「專家判斷」變成「可重現測量」。

3.2 Tier 2: Scout Sampling (Low Cost)

Deploy quasi-random sample points (Sobol sequence or Latin Hypercube) across the solution space, evaluate the objective at each point, and fit a surrogate model (e.g., Gaussian Process) to approximate the full landscape.

Outputs:

1. Estimated loss value at any unvisited point.
2. Uncertainty estimate (high near sample points, low far away).
3. Landscape roughness index (coefficient of variation of sampled losses).
4. Basin count and approximate basin locations.

Cost: 20–50 function evaluations. Applicable to Boxes 1, 2 (partially), 4, 5 (partially), and 6.

Limitation: Surrogate accuracy degrades on rugged or deceptive landscapes where neighbouring solutions have vastly different objective values.

3.3 Tier 3: Mass Sampling (Medium Cost)

For landscapes where scout sampling fails (rugged, deceptive), deploy large-scale random sampling and analyse the statistical distribution of objective values rather than fitting a surrogate.

Outputs:

1. Distribution of objective values (skewness, kurtosis).

2. Autocorrelation of objective values along random walks (landscape smoothness).

3. Fitness-distance correlation (correlation between solution quality and distance to best-known solution).

Cost: 100–1000 function evaluations. Applicable to all boxes, particularly Boxes 2c, 3a (near phase transition), 5c.

3.4 Reconnaissance Coverage

Box	Aerial	Scout	Mass
1 (Combinatorial)	✓	✓	—
2 (Graph)	✓	partial	✓
3 (Logic)	✓	—	✓
4 (Path/Network)	✓	✓	—
5 (Permutation)	✓	partial	✓
6 (Sequential)	✓	✓	—
7 (Miscellaneous)	case-by-case	case-by-case	case-by-case

All six main boxes can be reconnoitred by at least one tier. No box is invisible.
所有六個主要盒子至少可以被一個偵察層級覆蓋。沒有任何盒子是看不見的。

3.5 Statistical Foundations of Reconnaissance

Reconnaissance is a statistical inference problem: given a finite sample of the landscape, decide which category the full landscape belongs to. Like any statistical test, it is subject to error. This section provides the formal framework for controlling these errors and determining the appropriate reconnaissance investment.

偵察是一個統計推論問題：根據曲面的有限樣本，判斷完整曲面屬於哪一類。與任何統計檢定一樣，它存在誤差。

3.5.1 Type I and Type II Errors in Reconnaissance

In the reconnaissance context:

- **Type I error (false positive):** Scouts report "landscape is simple" when it is actually difficult → lightweight algorithm deployed → poor solution quality. *This is the dangerous error.*

- **Type II error (false negative):** Scouts report "landscape is difficult" when it is actually simple → heavy-duty algorithm deployed → correct solution but wasted computation. *This is costly but not fatal.*

Error rates vary by reconnaissance tier:

Tier	Type I (dangerous)	Type II (wasteful)
Aerial survey (zero cost)	$\sim$15–20%	$\sim$30%
Scout sampling (50 points)	$\sim$5%	$\sim$15%
Mass sampling (1000 points)	$< 2\%$	$\sim$5%

The framework is inherently biased toward Type II errors: it prefers to overestimate difficulty rather than underestimate it. This is a deliberate design choice analogous to the NSD trading system's defence-first philosophy—the stop-loss net catches worst-case scenarios even when the main strategy fails.

框架天然偏向型二誤差：寧可高估難度多花算力，也不要低估難度解錯。這與 NSD 交易系統的防守優先哲學一致。

3.5.2 Power Analysis for Scout Count

The number of scout samples is not arbitrary. It is determined by statistical power analysis:

- **What to detect:** whether the landscape is unimodal or multimodal (the effect).
- **Acceptable Type I error:** α (e.g., 0.05).
- **Required power:** $1 - \beta$ (e.g., 0.90 = 90% detection probability).
- **Effect size:** the magnitude of the difference between basins (estimated from aerial survey).

The power calculation yields the minimum number of scout points N needed to achieve the desired detection probability. Smoother landscapes (large effect size) require fewer scouts; rugged landscapes (small effect size) require more.

The progressive structure is:

1. Aerial survey $\rightarrow$ rough estimate of effect size (zero cost).
2. Power test $\rightarrow$ required number of scouts N (one calculation, zero cost).
3. Deploy exactly N scouts $\rightarrow$ not more, not less.
4. Scouts determine algorithm and parameters.

This ensures that even the reconnaissance investment is quantitatively optimised. The computational budget saved by the framework (Section 8) provides the headroom for higher-power reconnaissance when needed.

power 分析確保偵察投資本身也是量化最佳化的。較平滑的曲面需要較少斥候，較崎嶇的需要較多。空中偵查先粗估 effect size（零成本），power test 算出所需斥候數量（零成本），然後精確部署。

3.5.3 Multiple Testing Correction

The reconnaissance system simultaneously evaluates four difficulty scores (ruggedness, multimodality, deceptiveness, feasibility). Testing four hypotheses simultaneously inflates the overall Type I error rate. If each individual test uses $\alpha = 0.05$, the family-wise error rate is approximately $1 - (1 - 0.05)^4 \approx 0.185$—nearly four times the intended level.

We apply Bonferroni correction: each individual threshold is set at $\alpha/4 = 0.0125$, maintaining the overall Type I error rate at 0.05. This is conservative (may increase Type II errors), which is consistent with the framework's defence-first design.

Remark 5 (Correlated scores and alternative corrections)**.** *The four difficulty scores are computed from a shared pool of reconnaissance samples and are therefore not statistically independent. Bonferroni correction is valid but conservative under correlation: it still controls the family-wise error rate at α, but may over-correct when the true correlation structure is known to be positive. More powerful alternatives include Holm–Bonferroni (step-down procedure) and the Benjamini–Hochberg procedure (controlling false discovery rate rather than family-wise error), both of which remain valid under positive dependence. We adopt plain Bonferroni in this paper because it is the most conservative and requires no assumption about the correlation structure; users with empirical knowledge of the score correlations on their problem family may substitute Holm–Bonferroni for improved power.*

同時檢定四個難度分數會導致整體型一誤差膨脹。使用 Bonferroni 校正（每個閾值設為 $\alpha/4$）維持整體型一誤差在 0.05。此校正偏保守，符合框架的防守優先設計。四個難度分數來自共享的偵察樣本池，因此在統計上不獨立；Bonferroni 校正在相關檢定下仍有效，但偏保守。若使用者對分數相關性有經驗知識，可改用 Holm–Bonferroni 或 Benjamini–Hochberg 程序以提升檢定力。

3.5.4 Effect Size Estimation

Power analysis requires an effect size, but the effect size is unknown before reconnaissance—a chicken-and-egg problem. The solution is to use the aerial survey's structural indicators as a rough proxy:

- **Box 1 (Combinatorial):** Item weight/value variance → proxy for basin depth difference.
- **Box 2 (Graph):** Degree distribution spread → proxy for landscape ruggedness.
- **Box 3 (Logic):** Distance of α from 4.26 → proxy for phase transition proximity.
- **Box 4 (Path):** Spatial clustering index → proxy for number of basins.
- **Box 5 (Permutation):** Hard constraint ratio → proxy for feasible region width.
- **Box 6 (Sequential):** κ^* convergence rate → proxy for landscape learnability.

These proxies need not be precise. In practical power analysis applications, moderate misspecification of effect size typically inflates the required sample size by a bounded factor rather than invalidating the calculation: a proxy within roughly a factor of two of the true effect size usually yields a scout count of the correct order of magnitude, sufficient for the framework's defence-first preference for Type II over Type I errors. The required precision is order-of-magnitude, not exact.

Effect size 估計不需要精確。在實務 power 分析應用中，effect size 的中等誤差通常只使所需樣本量在有界範圍內膨脹，而非使計算失效：proxy 與真實 effect size 相差約兩倍以內時，通常能得出正確數量級的斥候數量，這對框架「寧可多花算力（型二誤差）也不要判錯地形（型一誤差）」的防守優先偏好已經足夠。所需精度為數量級等級，非精確值。

3.5.5 Surrogate Uncertainty Propagation

When Tier 2 scout sampling fits a Gaussian Process surrogate, the GP provides both a predicted loss $\mu(\mathbf{x})$ and a prediction uncertainty $\sigma(\mathbf{x})$ at every point. This uncertainty should be propagated into the four difficulty scores via *inverse-variance weighting*: let $s_1, s_2, \ldots, s_N$ denote per-point contributions to a given difficulty score, and let σ_i^2 denote the GP predictive variance at point i. The uncertainty-weighted aggregate score is

$$\hat{S} = \frac{\sum_{i=1}^{N} s_i/\sigma_i^2}{\sum_{i=1}^{N} 1/\sigma_i^2}, \tag{23}$$

with associated standard error $\mathrm{SE}(\hat{S}) = (\sum_i 1/\sigma_i^2)^{-1/2}$. The practical effect is as follows:

- Points far from any scout sample have high $\sigma_i^2 \to$ their contribution to the difficulty scores is automatically down-weighted by the $1/\sigma_i^2$ factor.
- Points near multiple samples have low $\sigma_i^2 \to$ their contribution receives high weight and is treated as reliable.

If the average GP uncertainty across the landscape exceeds a threshold (equivalently, if $\mathrm{SE}(\hat{S})$ is too large to distinguish difficulty regimes), the power analysis indicates that additional scout points are needed. This creates a feedback loop: GP uncertainty $\to$ insufficient power $\to$ deploy more scouts $\to$ reduce GP uncertainty $\to$ re-evaluate. The loop terminates when $\mathrm{SE}(\hat{S})$ is below the required threshold to achieve the desired power level.

當 Tier 2 斥候採樣擬合高斯過程代理模型時，GP 在每個點同時提供預測損失 $\mu(\mathbf{x})$ 與預測不確定性 $\sigma(\mathbf{x})$。此不確定性應以*反變異數加權*傳播至四個難度分數（見式 23）：遠離斥候樣本的點 σ_i^2 高，其貢獻被 $1/\sigma_i^2$ 自動降權；鄰近多個樣本的點 σ_i^2 低，其貢獻獲高權重。若整體 landscape 的平均 GP 不確定性超過閾值（等價於 $\mathrm{SE}(\hat{S})$ 太大而無法區分難度等級），power 分析指出需要追加斥候。此形成一個反饋迴圈，直到 $\mathrm{SE}(\hat{S})$ 低於所需閾值為止。

The computational cost of all statistical operations described in this section—Bonferroni correction (one division), effect size estimation (one formula substitution), power calculation (one function evaluation), and GP uncertainty check (already computed by GP)—is effectively zero. These are arithmetic operations on data that already exists from the reconnaissance phase.

本節所述所有統計運算的計算成本趨近零：Bonferroni 校正（一次除法）、effect size 估計（一次公式代入）、power 計算（一次函數評估）、GP 不確定性檢查（GP 已算好）。全部都是在偵察階段已有數據上的算術運算，不需要新的函數評估。

4 The Five-Cut Decision Tree

Given a classified problem and its reconnaissance data, the following decision tree determines the optimal algorithm family.

1. **Cut 1: Box assignment.** Which of the seven boxes does the problem belong to? This determines the building method for the loss landscape.
2. **Cut 2: Neighbourhood / landscape construction.** Within the box, which neighbourhood definition or relaxation method is used? This determines the landscape's adjacency structure.

3. **Cut 3: Continuous or discrete?** Is the resulting landscape continuous (amenable to gradient methods) or discrete (requiring combinatorial search)?

4. **Cut 4: Landscape topology.**

   - Unimodal → gradient descent / local search
   - Multimodal with structure → memetic algorithm (GA + local search)
   - Extremely rugged → large-population GA / random restart
   - Flat plateau → simulated annealing / random restart
   - Deceptive → large-population GA / EDA

5. **Cut 5: Special properties.**

   - Heavy constraints? → Penalty functions / Lagrangian relaxation / Constraint Programming
   - Dynamic state collapse? → MCCO / Reinforcement Learning
   - Analytic Jacobians available? → BPTT / gradient-based methods

五刀切完後，演算法直接確定。有些問題第二刀就分類完畢（例如背包用 DP），有些需要切到第五刀（例如帶約束的動態序列決策用 MCCO）。

5 Landscape Types and Algorithm Matching

Landscape Type	Characteristics	Best-Fit Algorithms
Continuous unimodal	Single basin, gradient available	Gradient descent, BPTT, Newton
Continuous multimodal	Multiple basins, gradient available	Multi-start gradient, MCCO, Basin Hopping
Continuous rugged	Many small pits, noisy gradient	SGD, smoothing + gradient, SA
Discrete smooth	Neighbours similar, clear trend	Local search, Hill Climbing, Tabu
Discrete multimodal	Multiple basins, structured ridges	GA + local search, Ant Colony
Discrete rugged	Neighbours may differ vastly	Large-population GA, random restart
Flat plateau	Large flat regions, hidden valleys	Simulated Annealing, random restart
Deceptive	Local optima mislead	Large-population GA, EDA
Constrained	Narrow feasible region, cliffs	Penalty, Lagrangian, CP
Dynamic collapse	Landscape changes with observation	MCCO, Reinforcement Learning

Remark 6 (Default vs Confirmed)**.** *The algorithm assignments in this table are* default recommendations*—the best choice in the absence of reconnaissance data. After scout sampling or aerial survey, the confirmed assignment may differ from the default. The following two sections*

exhaustively enumerate how reconnaissance results can modify both algorithm selection and parameter configuration.

上表中的演算法配置為預設建議——在沒有偵察資料的情況下的最佳選擇。斥候偵察或空中偵查之後，確認的配置可能與預設不同。以下兩節窮舉了偵察結果如何修改演算法選擇和參數配置。

6 Scout-Based Algorithm Adjustment

For each sub-box, the default algorithm is the best choice when no landscape information is available. After reconnaissance, the following adjustments may apply. This exhaustive enumeration ensures that no sub-box is left without guidance.

對每個子盒子，預設演算法是無地形資訊時的最佳選擇。偵察之後可能需要以下調整。

6.1 Box 1: Combinatorial Optimisation

1a Knapsack. Default: DP. Adjustment: when $N \times W > 10^7$, DP memory explodes $\rightarrow$ switch to Greedy + LP relaxation.

1b TSP. Default: GA + 2-opt. Adjustment: cities highly clustered $\rightarrow$ unimodal landscape $\rightarrow$ pure local search suffices; cities extremely uniform $\rightarrow$ severe multimodality $\rightarrow$ increase GA population.

1c Bin Packing. Default: FFD + SA. Adjustment: item sizes nearly identical $\rightarrow$ plateau landscape $\rightarrow$ SA with high initial temperature and slow cooling; item sizes vary greatly $\rightarrow$ obvious good solutions $\rightarrow$ greedy suffices.

1d Cutting Stock. Default: Column Generation. Adjustment: very few demand types $\rightarrow$ simple DP suffices for subproblems.

6.2 Box 2: Graph Theory

2a Graph Colouring. Default: Tabu Search. Adjustment: very sparse graph $\rightarrow$ greedy suffices; very dense graph $\rightarrow$ Constraint Programming better handles abundant constraints.

2b-i Vertex Cover / Set Cover. Default: Greedy (2-approximation for Vertex Cover, $\ln n$ for Set Cover). Adjustment: exact solution needed $\rightarrow$ Branch and Bound with LP bound; tree-structured graph $\rightarrow$ linear-time exact solution exists; Set Cover with small universe $\rightarrow$ ILP is tractable.

2b-ii Dominating Set. Default: Greedy (logarithmic approximation). Adjustment: planar or bounded-treewidth graph $\rightarrow$ fixed-parameter tractable algorithms; small instances $\rightarrow$ ILP; general instances $\rightarrow$ no constant-factor approximation is known, accept logarithmic gap.

2c Maximum Clique. Default: Branch and Bound. Adjustment: medium-density graph (most deceptive) $\rightarrow$ large-population GA; extreme density (high or low) $\rightarrow$ B&B pruning is effective, use directly.

2d Steiner Tree. Default: Approximation algorithm. Adjustment: very few terminal nodes (< 15) $\rightarrow$ DP exact solution feasible.

6.3 Box 3: Logical Satisfaction

3a 3-SAT. Default: CDCL. Adjustment (aerial survey suffices): $\alpha \ll 4.26 \rightarrow$ WalkSAT is faster; $\alpha \approx 4.26 \rightarrow$ CDCL is the only viable option.

3b CSP. Default: Backtracking + AC. Adjustment: constraints extremely tight → CP with enhanced propagation; constraints very loose → local search suffices.

3c MAX-SAT. Default: SA. Adjustment: few clauses → Branch and Bound for exact solution.

6.4 Box 4: Path and Network

4a Hamiltonian Path. Default: GA + local search. Adjustment: high connectivity → local search alone suffices; low connectivity → multiple random restarts needed.

4b VRP. Default: LNS. Adjustment: customers naturally clustered → cluster-first route-second (decompose then solve each cluster as TSP); very tight time windows → Constraint Programming more suitable.

4c Facility Location. Default: LP + B&B. Adjustment: very few facilities → enumeration feasible; LP relaxation integrality gap very small → LP solution rounded directly.

6.5 Box 5: Permutation and Assignment

5a Nurse Scheduling. Default: CP. Adjustment: few hard constraints → Tabu Search faster; many hard constraints with extremely narrow feasible region → CP with symmetry breaking.

5b Weighted Matching. Default: Hungarian Algorithm. Already P; no adjustment typically needed.

5c QAP. Default: SA. Adjustment: flow and distance matrices highly correlated → structured landscape → GA + local search may outperform; completely unstructured → large-population GA ($P > 300$).

6.6 Box 6: Sequential Decision

6a Financial. Default: MCCO. Adjustment: algorithm unchanged; scout determines chain count and per-chain configuration (unimodal → 2 chains; multimodal → 5+ chains).

6b Medical. Default: MCCO variant. Adjustment: purely discrete problem (no continuous parameters) → GA alone suffices without BPTT.

6c Control. Default: MCCO. Adjustment: no analytic Jacobians available → degrade to Reinforcement Learning.

6.7 Key Observation

Most adjustments do not cross algorithm families—they modify configuration within the same family (population size, cooling schedule, restart strategy). Only a minority of cases trigger a complete algorithm switch. This confirms that the default recommendations are robust starting points, with scouts providing fine-tuning rather than fundamental redirection.

大多數調整不會跨越演算法族——而是在同一族內修改配置。僅少數情況觸發完全的演算法切換。這確認了預設建議是穩健的起點，斥候提供的是微調而非根本性改向。

7 Landscape-Based Parameter Configuration

Beyond algorithm selection, reconnaissance data determines the optimal *parameter settings* for the chosen algorithm. This section provides a complete mapping from landscape features to parameter values.

除了演算法選擇外，偵察數據還決定所選演算法的最佳參數設定。

7.1 Genetic Algorithm Family

Landscape	Population	Mutation Rate	Crossover
Unimodal smooth	20–30	5%	Standard
Multimodal structured	50–100	10–20%	Standard
Rugged / deceptive	200+	30%+	Uniform (preserve diversity)

7.2 Simulated Annealing

Landscape	Initial Temp	Cooling Rate	Special
Smooth	Low	Fast	—
Plateau	High	Very slow	Long exploration phase
Multimodal	High	Medium	Add reheating

7.3 Local Search / Tabu Search

Landscape	Neighbourhood	Tabu List	Special
Smooth	Small	Short (5–10)	—
Rugged	Large	Long (20+)	—
Plateau	Any	Any	Add random restart

7.4 Gradient-Based / BPTT

Landscape	Learning Rate	Paths (M^*)	Special
Steep narrow valley	Small (0.001)	Standard	—
Flat plateau	Large (0.01)	Standard	—
High noise	Standard	Increase M^*	Average out stochasticity

7.5 Branch and Bound

Landscape	Strategy	Special
LP gap small	Direct B&B	Pruning effective
LP gap large	B&B + cutting planes	Gomory cuts / Benders

7.6 SAT Solvers

Landscape (α)	Solver	Noise / Restart	Special
$\alpha \ll 4.26$	WalkSAT	noise = 0.3–0.5	—
α slightly below 4.26	WalkSAT	noise = 0.5–0.7	More exploration
$\alpha \approx 4.26$	CDCL	Frequent restarts	Clause deletion strategy

7.7 Sub-Box Specific Configurations

The following provides the complete parameter lookup for each sub-box after reconnaissance:

1a Knapsack. DP: table size $N \times W$; if $> 10^7$, switch. No parameter tuning needed.

1b TSP. GA: clustered cities $\rightarrow P = 30$, mutation $= 10\%$; uniform cities $\rightarrow P = 100$, mutation $= 20\%$. ACO: smooth landscape $\rightarrow$ evaporation $= 0.5$ (fast convergence); multimodal $\rightarrow$ evaporation $= 0.1$ (retain route memory).

1c Bin Packing. SA: large size variance $\rightarrow$ low initial temp, fast cooling; small variance $\rightarrow$ high initial temp, slow cooling.

1d Cutting Stock. Column Generation: many demand types $\rightarrow$ DP subproblem; few types $\rightarrow$ enumeration subproblem.

2a Graph Colouring. Tabu: sparse $\rightarrow$ list $=$ 5–10, small neighbourhood; dense $\rightarrow$ list $=$ 20+, large neighbourhood. CP: dense $\rightarrow$ upgrade AC-3 to AC-4.

2b-i Vertex Cover / Set Cover. Greedy: no tuning. B&B: sparse $\rightarrow$ depth-first; dense $\rightarrow$ add LP bound. Set Cover ILP: small universe (< 100) $\rightarrow$ direct ILP; larger $\rightarrow$ LP rounding.

2b-ii Dominating Set. Greedy: no tuning. FPT: bounded treewidth $\rightarrow$ dynamic programming on tree decomposition; planar $\rightarrow$ bidimensionality theorem applies.

2c Maximum Clique. GA: medium density $\rightarrow P = 200$+, mutation $= 30\%$, uniform crossover. B&B: extreme density $\rightarrow$ strong pruning, use directly.

2d Steiner Tree. DP: terminals $< 15 \rightarrow$ exact. Approximation: terminals $\geq 15 \rightarrow$ 2-approximation.

3a 3-SAT. CDCL: $\alpha \approx 4.26 \rightarrow$ aggressive clause deletion + frequent restarts. WalkSAT: $\alpha \ll 4.26 \rightarrow$ noise $= 0.3$–0.5; α slightly below $4.26 \rightarrow$ noise $= 0.5$–0.7.

3b CSP. Backtracking: tight constraints $\rightarrow$ MRV variable ordering + LCV value ordering; loose constraints $\rightarrow$ random ordering suffices.

3c MAX-SAT. SA: many clauses $\rightarrow$ high temp, slow cooling; few clauses $\rightarrow$ B&B exact.

4a Hamiltonian Path. GA: high connectivity $\rightarrow P = 30$, mutation $= 10\%$; low connectivity $\rightarrow P = 100$, mutation $= 20\%$ + random restarts.

4b VRP. LNS: clustered customers $\rightarrow$ destroy $= 10$–15%; dispersed $\rightarrow$ destroy $= 30$–40%. ACO: tight time windows $\rightarrow$ high heuristic weight β; loose $\rightarrow$ high pheromone weight α.

4c Facility Location. LP+B&B: small integrality gap $\rightarrow$ LP round directly; large gap $\rightarrow$ add Gomory cuts or Benders decomposition.

5a Nurse Scheduling. CP: many hard constraints $\rightarrow$ symmetry breaking + enhanced propagation; few hard constraints $\rightarrow$ basic backtracking. Tabu: many hard constraints $\rightarrow$ long list + heavy violation penalty; few $\rightarrow$ short list.

5b Matching. Hungarian: $O(N^3)$ fixed, no tuning.

5c QAP. SA: structured matrices $\rightarrow$ medium temp, medium cooling; unstructured $\rightarrow$ high temp, very slow cooling + reheating. GA: unstructured $\rightarrow P = 300$+, mutation $= 30\%$+, OX crossover for permutation legality.

6a Financial (MCCO). Verified: unimodal $\rightarrow$ 2 chains, standard LR; multimodal $\rightarrow$ 5+ chains; high Lot sensitivity $\rightarrow$ Lot LR $\times 0.5$.

6b Medical. MCCO variant: more discrete components $\rightarrow$ increase GA weight, mutation $= 20\%$; more continuous components $\rightarrow$ increase BPTT weight.

6c Control. MCCO with Jacobians $\rightarrow$ standard config. RL without Jacobians $\rightarrow$ PPO, LR $= 3 \times 10^{-4}$, batch $= 64$ (simple) to 256 (complex).

以上為 21 個主要子盒子（盒子 1–6）的完整參數查詢表。偵察結果決定山形，山形決定參數。整個流程在開始解題之前就全部完成——還沒開始爬山，就已經知道穿什麼鞋、帶多少繩、從哪個方向走、走多快。

8 Landscape Transformation

After reconnaissance, some landscapes may be too difficult to attack directly—too rugged, too deceptive, or too constrained. Rather than accepting the landscape as given, we can *transform* it into a simpler one before deploying the solver. This is analogous to reshaping a mountain into a gentler slope before climbing.

偵察之後，某些曲面可能太難直接攻克。與其接受原始地形，不如先將其變形為更簡單的曲面再部署演算法。

8.1 Transformation Methods

Individual transformation techniques are well-established across different fields, though they are rarely discussed under a unified name. We catalogue them here as **Landscape Transformations**:

Method	What It Does	Applicable Boxes
LP Relaxation	Discrete → continuous; stairs → slopes	1, 4, 5
Lagrangian Relaxation	Constraint cliffs → steep slopes (constraints moved into objective)	1, 4, 5
Smoothing (Gaussian convolution)	Rugged spikes → smooth surface	1, 2, 3
Convexification	Non-convex → convex (single basin)	Problem-specific
Change of Variables	Distorted valleys → round basins (coordinate transform)	All continuous
Embedding	Low-dim discrete → high-dim continuous (e.g. Sinkhorn)	2, 5
Spectral Relaxation	Discrete graph → continuous eigenvalue problem	2
Penalty Method	Constrained → unconstrained (constraints become steep slopes)	All constrained

8.2 Box-Specific Transformation Applicability

Box	Available Transformations	Effectiveness
1 (Combinatorial)	LP relaxation, Lagrangian, Penalty	High
2 (Graph)	Spectral relaxation, Embedding, Smoothing	High
3 (Logic)	Continuous relaxation, Smoothing	Limited near phase transition
4 (Path/Network)	LP relaxation, Lagrangian	High
5 (Permutation)	Sinkhorn embedding, Penalty	High
6 (Sequential)	Collapse chain (already transformed)	N/A (native)

Five and a half of the six boxes can be effectively transformed; only Box 3 near the phase transition resists transformation. Box 6 does not need transformation because the collapse chain already provides a natively differentiable substrate.

六個盒子中五個半可以有效變形；僅盒子三在相變附近變形效果差。盒子六不需要變形，因為坍縮鏈本身就提供了原生可微的基底。

8.3 Difficulty Scoring

To determine whether transformation is needed, we define four landscape difficulty scores computed during reconnaissance. Each score has a box-specific threshold; exceeding the threshold triggers transformation.

為判斷是否需要變形，我們在偵察階段計算四個曲面難度分數。每個分數有各盒子專屬的閾值，超過閾值即觸發變形。

Score	Definition	Range	Transform If
Ruggedness	Autocorrelation of losses along random walk	0 (rugged) to 1 (smooth)	$< \tau_{\text{rug}}$
Multimodality	Number of distinct basins from multi-start local search	1 to ∞	$> \tau_{\text{mod}}$
Deceptiveness (FDC)	Fitness-distance correlation	-1 (normal) to $+1$ (deceptive)	$> \tau_{\text{dec}}$
Feasibility	Fraction of random samples that are feasible	0 to 1	$< \tau_{\text{feas}}$

Remark 7 (Relative thresholds)**.** *These scores are relative, not absolute. The same ruggedness value may indicate a smooth landscape in one box and a rugged landscape in another. Therefore, thresholds τ must be calibrated* per box *using benchmark instances. Cross-box comparison of raw scores is not meaningful; within-box comparison is.*

這些分數是相對性的。同一個粗糙度數值在不同盒子裡可能代表不同意義，因此閾值 τ 必須按盒子分別校準。跨盒子比較原始分數無意義，盒子內比較有意義。

Remark 8 (Error architecture: all inference risk is in reconnaissance)**.** *The transformation and cutting decisions described in the following sections are* deterministic functions *of the difficulty scores: if a score exceeds τ, transform; if the evaluation matrix ranks cell A below cell B, select A. These downstream operations introduce no additional statistical error. All inference risk in the framework is concentrated in the reconnaissance layer, where it is controlled by the Type I/II framework of Section 3 (power analysis, Bonferroni correction, GP uncertainty propagation). Reconnaissance is inference; transformation and cutting are execution.*

變形與切割決策是難度分數的確定性函數——分數超過閾值就變形，矩陣排名最低就選最低。這些下游操作不引入額外的統計誤差。框架的全部推論風險集中在偵察層，由 Section 3 的統計框架控制。偵察是推論，變形和切割是執行。

However, the scores themselves are estimated from finite reconnaissance samples and therefore carry sampling uncertainty. This residual uncertainty propagates into the evaluation matrix: when two cells have similar scores, the ranking may be reversed by noise. Section 10.5 addresses this with a score significance test.

但分數本身由有限偵察樣本估計，因此帶有抽樣不確定性。此殘餘不確定性會傳播到評估矩陣：當兩格分數接近時，排名可能被噪音翻轉。Section 10.5 以分數顯著性檢定處理此問題。

8.4 The Reconnaissance–Transformation Loop

The complete pre-solving pipeline is a closed loop:

1. **Classify**: Assign the problem to a box.
2. **Reconnoitre**: Compute the four difficulty scores.
3. **Evaluate**: Are all scores within thresholds?
   - **Yes** $\rightarrow$ proceed to algorithm selection and parameter configuration (Section 6–7).
   - **No** $\rightarrow$ apply the most appropriate transformation for the offending score.
4. **Re-reconnoitre**: Compute scores on the transformed landscape.
5. **Repeat** from Step 3 until a stopping condition is triggered.

8.5 Four-Layer Stopping Conditions

The loop must terminate. We define four stopping conditions, checked in order after each transformation:

1. **Score below threshold** (success). All four difficulty scores fall below their box-specific thresholds. The landscape is now simple enough for efficient solving. Proceed to algorithm selection.
2. **Marginal benefit below threshold** (diminishing returns). Let Δ_k denote the score reduction achieved by the k-th transformation. If $\Delta_k / \Delta_{k-1} < \epsilon$ (e.g., $\epsilon = 0.1$), the current transformation achieves less than 10% of the previous one's improvement. Further transformation is not worthwhile. Accept the current landscape and select an algorithm matched to its (higher) difficulty level.

   This condition handles the edge case where each transformation produces a tiny improvement (e.g., 0.00001 per iteration) that technically satisfies "still improving" but provides no practical benefit. By measuring the *ratio* of consecutive improvements rather than their absolute value, the system detects diminishing returns and stops.
3. **Cumulative transformation cost exceeds direct solving cost** (negative ROI). Before each transformation, estimate:
   - $C_{\text{transform}}$: cost of the next transformation (estimated from problem size; e.g., LP relaxation cost scales with variables $\times$ constraints).
   - $C_{\text{solve}}^{\text{current}}$: estimated cost of solving at the current difficulty score (looked up from benchmark tables indexed by score and problem size).
   - $C_{\text{accumulated}}$: total cost already spent on reconnaissance and transformations.

   If $C_{\text{accumulated}} + C_{\text{transform}} + C_{\text{solve}}^{\text{after}} > C_{\text{solve}}^{\text{original}}$, the total cost of transforming exceeds the cost of solving the original landscape directly. Stop and solve the original (or best-so-far transformed) landscape.

   These cost estimates are order-of-magnitude approximations, not exact values. Transformation costs are estimated from problem size (deterministic). Solving costs are estimated from difficulty scores plus historical benchmark data for similar-sized instances. The comparison need only be accurate to within an order of magnitude to make correct decisions.

4. **Maximum iteration limit** (safety net). A hard upper bound (e.g., 10 iterations) prevents the loop from running indefinitely in pathological cases where marginal benefit oscillates near the threshold.

四層停止條件：(1) 分數低於閾值 → 成功；(2) 邊際效益低於閾值 → 繼續轉沒意義；(3) 累計變形成本超過直接求解成本 → 轉不如不轉；(4) 最大次數上限 → 安全網。成本估計是量級估計而非精確值：變形成本由問題規模決定（確定性的），求解成本由偵察分數 + 歷史基準數據查表估計。量級對就能做出正確決策。

Remark 9 (Failing to reach the threshold is not failure)**.** *If the loop terminates at Condition 2, 3, or 4 with scores still above the threshold, the problem is not abandoned. The current (partially transformed) landscape is classified at its actual difficulty level, and a solver appropriate for that difficulty is deployed. An 80-point rugged landscape that could only be reduced to 75 is solved with heavy-duty methods (large-population GA, random restart); a 40-point smooth landscape is solved with lightweight methods (local search). The framework provides a matched solver for* every *difficulty level, not only for easy ones.*

分數未達閾值不代表失敗。迴圈因條件 2、3 或 4 停止時，以當前實際難度分數分類曲面，配置對應難度的演算法。框架為每個難度等級都提供匹配的求解器，不僅限於簡單的。

8.6 Transformation as Computational Investment

Landscape transformation is a computational investment: spend a small amount upfront to reduce the cost of the main solve. This is identical in principle to the reconnaissance investment demonstrated in the MCCO proof of concept (Section 12), where 1.4 seconds of scout sampling saved 4.3 seconds of solver runtime.

At the framework level, transformation amplifies this effect. An 80-point rugged landscape might require hours of large-population GA. Spending 2 minutes on LP relaxation to reduce it to a 50-point multimodal landscape allows a memetic algorithm to solve it in minutes. The 2-minute investment yields a multi-fold return.

The two layers of acceleration—reconnaissance (choose the right algorithm) and transformation (simplify the landscape)—compound multiplicatively. Together, they represent the framework's core mechanism for narrowing the practical gap between NP and P: not by proving theoretical equivalence, but by engineering away the computational overhead that makes NP-hard problems slow in practice.

曲面變形是一種計算投資：花少量前置成本降低主要求解的成本。偵察（選對演算法）和變形（簡化曲面）兩層加速以乘法方式疊加。這是本框架縮小 NP 與 P 之間實務差距的核心機制：不是證明理論等價，而是工程地消除讓 NP 難題在實務上變慢的計算開銷。

8.7 Precision Recovery

After solving on the transformed landscape, a *refinement step* maps the solution back to the original landscape and applies local search to recover precision lost during transformation. This is standard practice in LP relaxation (rounding + local improvement) and spectral methods (discretisation + refinement).

8.8 Updated Complete Pipeline

Incorporating landscape transformation and the four-layer stopping conditions, the full framework pipeline becomes:

1. **Cut 1**: Assign to box (problem type).
2. **Cut 2**: Choose neighbourhood / construction method.
3. **Reconnoitre**: Three-tier reconnaissance (aerial $\rightarrow$ scout $\rightarrow$ mass).
4. **Score**: Compute four difficulty scores.
5. **Transform loop**: While any score exceeds threshold *and* no stopping condition is triggered, apply transformation and re-score.
6. **Cut 3–5**: Classify (possibly transformed) landscape (continuous/discrete, topology, special properties).
7. **Select algorithm**: Match landscape type to solver family.
8. **Configure parameters**: Set solver parameters based on landscape features.
9. **Solve**: Deploy solver.
10. **Refine**: Map solution back to original space; local search for precision recovery.

This ten-step pipeline is the complete operational procedure for attacking any NP-hard problem using the proposed framework. No step requires trial and error; every decision is informed by quantitative landscape data. The two acceleration layers—reconnaissance and transformation—compound to minimise total computational cost, pushing the practical solving time of NP-hard problems as close to polynomial as the problem structure allows.

此十步流程是使用本框架攻克任何 NP 難題的完整操作程序。每一步決策都由量化的曲面數據驅動，無需試誤。偵察與變形兩層加速以乘法疊加，將 NP 難題的實務求解時間盡可能逼近多項式時間。

9 Landscape Cutting (Decomposition)

After transformation (if applied), a large landscape may benefit from being *cut* into smaller sub-landscapes, each solved independently and then reassembled. This is analogous to dividing a large mountain range into sectors, assigning a team to each sector, and coordinating results through a central command.

變形之後，大型曲面可以進一步*切割*為較小的子曲面，各自求解後再組合。

9.1 Cutting Principle

The fundamental requirement of cutting is: **complete coverage, no gaps, no orphaned fragments**. Like cutting a cake, every slice must be contiguous, and all slices together must reconstitute the original problem. Formally:

- The union of all blocks = the entire solution space (no omission).
- Each block is internally connected (no fragmented pieces).
- Cross-block interactions are minimised (cut along the thinnest connections).

9.2 Cutting Tools

Cuts need not be straight lines. The cut shape should follow the problem's *natural structure*—curved boundaries along natural gaps, not arbitrary straight divisions.

Cutting Tool	How It Cuts	Best For
K-means	Spherical clusters by distance	Geographic TSP, VRP
DBSCAN	Density-based, irregular shapes	Clustered data, natural gaps
Graph Partitioning	Minimise cross-partition edges	Graph problems (Box 2)
Geographic Grid	Regular grid cells	Spatially distributed problems
Variable Dependency	Group co-occurring variables	SAT, CSP (Box 3)

The choice of cutting tool is *not* predetermined—it is determined by the evaluation matrix (Section 10).

9.3 Box-Specific Cuttability

Not all problems benefit from cutting. The key factor is **sub-problem independence**—whether cutting destroys essential cross-block information.

Box	Cuttability	Reason
1 (Combinatorial)	High	Items/cities can be grouped
2 (Graph)	High	Graph partitioning is mature
3 (Logic)	Low	Variables highly entangled
4 (Path/Network)	High	VRP is naturally decomposable
5 (Permutation)	Medium	Cross-group constraints
6 (Sequential)	High	Chains already run in parallel

9.4 Transform First, Then Cut

The correct ordering is: **transform first, then cut**. After transformation, the landscape is smoother, natural gaps are more visible, and cuts are more precise. Cutting a rugged landscape risks cutting through basins; cutting a smoothed landscape follows natural contours.

正確順序：先變形再切割。變形後曲面更光滑，天然裂縫更清晰，切割更精確。

9.5 Parallel Deployment

After cutting, each block is an independent sub-problem:

1. Each block receives its own scout team → independent reconnaissance.
2. Each block is scored independently → may receive different algorithms.
3. All blocks solve in parallel → wall-clock time = slowest block.

4. Central command assembles results → global solution.

5. **Global refinement** → local search on the full assembled solution to repair boundary seams.

Global refinement after assembly is essential. Without it, boundary quality degrades by 7–11% (Section 13). With refinement, the cut-and-reassemble solution can match or even exceed the uncut solution, because cutting provides a better starting point for refinement.

全局精修不可或缺。不精修的切割品質下降 7–11%；精修後的切割 + 組裝方案可以追平甚至超越不切割的方案。

9.6 Five Cutting Considerations

1. **How many blocks?** Not fixed. Determined by problem size and cuttability. Too many → fragments too small, boundary costs explode. Too few → no speedup. Rule of thumb: 1 block per 10–50 entities, adjusted by reconnaissance.

2. **Equal-sized blocks?** Not necessarily. Simple regions can be large blocks; complex regions should be smaller blocks. Like military deployment: fewer troops where resistance is light, more where it is heavy.

3. **Recursive cutting?** Yes. If a block is still too difficult after cutting, cut it again. Large mountain → medium hills → small mounds. The recursion terminates when each block's difficulty score falls below threshold.

4. **Inter-block communication?** In the basic framework, blocks are independent until assembly. Advanced implementations could share boundary information during solving, at the cost of coordination overhead.

5. **Re-cutting?** If post-cut scoring reveals that a block's difficulty *increased* (e.g., a basin was split), the cut was wrong. Discard and try a different cutting tool or configuration. This is checked by the evaluation matrix.

10 The Evaluation Matrix

Rather than guessing whether to transform, which transformation to use, whether to cut, and which cutting tool to apply, the framework exhaustively evaluates all combinations and selects the best.

10.1 Matrix Construction

With T transformation methods (including "none") and C cutting methods (including "no cut"), the evaluation matrix has $T \times C$ cells:

	No Cut	**K-means**	**DBSCAN**	**Geographic**
No transform	?	?	?	?
LP relaxation	?	?	?	?
Smoothing (light)	?	?	?	?
Smoothing (heavy)	?	?	?	?
Penalty	?	?	?	?
⋮	⋮	⋮	⋮	⋮

In practice: 9 transformations $\times$ 4 cutting methods $=$ 36 cells. Each cell is filled by: apply the transformation, apply the cut, evaluate the difficulty score. Select the cell with the lowest score.

10.2 Cost

Each cell costs one transformation (reconnaissance-level) plus one scoring (four numbers, near-zero cost). For 36 cells on a 100-city TSP, the total matrix evaluation cost is approximately 2–3 seconds—negligible compared to the subsequent solving time.

10.3 Empirical Validation

On a 100-city TSP instance, we evaluated a 12-cell matrix (3 transforms $\times$ 4 cuts):

Configuration	Distance	Time
none + no cut	**883.91**	0.17 s
none + K-means (6 clusters)	898.03	0.29 s
none + density (54 clusters)	965.63	0.41 s
none + geographic (9 clusters)	947.39	0.23 s
smooth + no cut	**883.91**	0.12 s
smooth + K-means	898.03	0.24 s
smooth + density	954.73	0.44 s
smooth + geographic	947.39	0.20 s
penalty + no cut	**883.91**	0.12 s
penalty + K-means	898.03	0.40 s
penalty + density	947.56	0.39 s
penalty + geographic	946.25	0.34 s

The matrix correctly identified "no cut" as the best strategy (this instance has no natural clusters). Density-based cutting produced 54 clusters from 100 cities—severe fragmentation—and scored worst. The matrix detected this automatically in 3.37 seconds total.

矩陣在 3.37 秒內正確識別出「不切割」為最佳策略。密度切割把 100 個城市切成 54 塊——嚴重碎片化——得分最差。矩陣自動偵測到這一點。

10.4 Cutting Quality Check

After cutting, each block is re-scored. If any block's difficulty score is *higher* than the pre-cut whole, the cut damaged the landscape. The matrix handles this automatically: the "no cut" option always exists as a baseline, and any cut that worsens scores will rank below it.

Remark 10 (Exhaustibility guarantee)**.** *The matrix is exhaustively enumerable because both dimensions are finite: transformation methods (at most 9) and cutting tools (at most 4). The total number of cells (at most 36) is small enough to evaluate completely at reconnaissance-level cost. No heuristic selection is needed;* ***if it can be exhausted, exhaust it****.*

矩陣可以窮舉，因為兩個維度都是有限的。能窮舉就不要猜。

10.5 Score Significance Testing

Exhaustive enumeration eliminates the need to *guess* which cell is best—every cell is evaluated. However, the difficulty scores that fill each cell are not exact values; they are estimates derived from finite reconnaissance samples (Sobol points, random walks, multi-start local search). Each score therefore carries sampling uncertainty.

When the top-ranked cell scores 45 and the second-ranked cell scores 47, the question is: *is this difference real, or is it noise?* If the confidence intervals overlap, the ranking may be an artefact of sampling variance, and the framework could be selecting a suboptimal configuration.

窮舉消除了「選哪格」的猜測，但每格的分數來自有限樣本，帶有抽樣不確定性。當第一名得 45 分、第二名得 47 分，差距可能只是噪音。

Protocol. After filling all $T \times C$ cells, perform a paired significance test between the top-ranked cell and the second-ranked cell:

1. Each cell's score is computed from n reconnaissance samples (e.g., $n = 50$ Sobol points). The score is a summary statistic (mean, autocorrelation, basin count) of these samples.

2. Compute the standard error of each score from the same samples. For autocorrelation-based ruggedness, bootstrap resampling of the random walk provides the standard error at negligible cost.

3. Apply a paired t-test (or Wilcoxon signed-rank test if normality is doubtful) to the score difference between the top two cells.

4. **If significant** ($p < 0.05$ after Bonferroni correction for multiple comparisons): the top-ranked cell is the confirmed winner. Deploy its configuration.

5. **If not significant** ($p \geq 0.05$): the two cells are statistically equivalent. Select whichever has lower computational cost (transformation + cutting overhead). If costs are also equivalent, select the simpler configuration (fewer transformations, fewer cuts).

Cost. The test requires only arithmetic on data already computed during matrix evaluation—no additional function evaluations. Bootstrap resampling of existing scout data adds at most a few milliseconds. The total cost of significance testing is effectively zero.

Why only the top two? If the top-ranked cell is significantly better than the second-ranked, it is *a fortiori* significantly better than all lower-ranked cells. Testing all $\binom{T \times C}{2}$ pairs is unnecessary.

檢定僅需對矩陣中已有的數據做算術運算，成本為零。只需檢定前兩名：若第一名顯著優於第二名，則對所有更低排名必然也顯著。

Remark 11 (Three-layer statistical discipline)**.** *The framework's statistical architecture is now complete across three layers:*

1. ***Reconnaissance layer*** *(Section 3): Type I/II error control via power analysis, Bonferroni correction, and GP uncertainty propagation. This controls the* accuracy *of scores.*

2. ***Exhaustion layer*** *(Section 10): Exhaustive enumeration of all transformation × cutting combinations. This eliminates* selection *error.*

3. ***Decision layer*** *(this section): Paired significance testing of top-ranked scores. This controls* ranking *error when score differences are small.*

Transformation and cutting themselves are deterministic operations that introduce no statistical error. The defence-first philosophy (bias toward Type II) pervades all three layers: reconnaissance overestimates difficulty, exhaustion never skips a cell, and score testing defaults to the simpler configuration when evidence is ambiguous.

框架的統計架構橫跨三層：(1) 偵察層控制分數的準確度；(2) 窮舉層消除選擇誤差；(3) 決策層控制分數接近時的排名誤差。變形和切割本身是確定性操作，不引入統計誤差。防守優先哲學貫穿三層：偵察高估難度、窮舉不跳格、分數接近時選最簡配置。

11 Algorithm Adequacy Scoring

The framework selects the correct algorithm family (Section 6) and configures its parameters (Section 7). However, a final check is needed: **is the selected algorithm strong enough for this problem?**

A 2-opt local search may be the correct algorithm *type* for a smooth TSP landscape, but if the problem has 1,000 cities, a basic 2-opt implementation is too weak to find good solutions regardless of parameter tuning.

11.1 Adequacy Score

We define the algorithm adequacy level as a discrete step function of problem scale and difficulty:

$$\text{Adequacy Level}(N, D) = \begin{cases} 1 & \text{if } N \times D < 5{,}000 \\ 2 & \text{if } 5{,}000 \leq N \times D < 50{,}000 \\ 3 & \text{if } 50{,}000 \leq N \times D < 200{,}000 \\ 4 & \text{if } N \times D \geq 200{,}000 \end{cases} \tag{24}$$

where N is the problem size and D is the aggregated difficulty score from reconnaissance (the sum or maximum of the four scores in Section 8, depending on whether the risk combination is additive or worst-case dominated). The function is deterministic and table-driven; the four regimes correspond to the minimum tool requirements shown below. The threshold values $5{,}000/50{,}000/200{,}000$ are calibrated from empirical benchmarks and should be adjusted per-box when cross-box calibration data becomes available.

$N \times D$	Level	Minimum Tool	Example
$< 5{,}000$	1	Greedy, basic local search	50-city smooth TSP
$5{,}000$–$50{,}000$	2	Multi-start 2-opt, SA	100-city TSP
$50{,}000$–$200{,}000$	3	NN + strong 2-opt + Or-opt	500-city TSP
$> 200{,}000$	4	GA + 3-opt / LKH + parallel	1000+ city TSP

If the selected algorithm's level is below the required level, the framework issues a warning: "tool insufficient, upgrade required." This prevents the failure mode observed in our 1,000-city test, where basic 2-opt was correctly selected as the algorithm *type* but was too weak in *implementation* for the problem scale.

如果所選演算法的等級低於要求等級，框架發出警告：「工具不足，需要升級」。這是零成本的檢查——問題規模已知，難度分數已算好，乘一下查表即可。

12 Proof of Concept: Box 6 and MCCO

Box 6 (Sequential Decision Under Uncertainty) has been fully unified by Memetic Collapse-Chain Optimisation (MCCO) [tonyrao, 2026a]. MCCO exploits the collapse chain [tonyrao, 2026b]—a sequential composition of Bayesian collapse operators on the probability simplex—as a common mathematical substrate for all sub-problems within Box 6.

The key structural advantages of the collapse chain that enable unification:

- **Analytic Jacobians**: exact gradient computation without numerical approximation.
- **Chain Forgetting Theorem**: $\rho < 1$ guarantees unconditional gradient stability.
- **Bounded trajectories**: $\gamma_t \in \Delta^{K-1}$ prevents divergence.

12.1 Landscape-Guided Deployment Result

Applying the reconnaissance framework (Tier 2 scout sampling with 50 Sobol points + Gaussian Process surrogate) to MCCO on 2026/04/10:

- Reconnaissance identified a unimodal loss surface (single basin).
- Guided deployment used 2 chains instead of 3.
- Result: identical six-parameter output, $3.56\times$ speedup (1.7 s vs 6.0 s).
- Reconnaissance cost: 1.4 s (negligible relative to total computation).

This demonstrates the framework's core value proposition: reconnaissance determines the minimum computational investment needed, eliminating waste on simple landscapes and increasing investment on complex ones.

Remark 12 (Two distinct speedups with different baselines)**.** *Two different speedup figures appear in this paper and should not be confused. The* $24{,}000\times$ *speedup reported for MCCO in Sub-box 6a (Section 2.8) refers to MCCO's core algorithmic acceleration relative to a baseline of Genetic Algorithm combined with Monte Carlo simulation (GA+MC)—a within-Box 6a comparison of two different algorithms on the same problem. The* $3.56\times$ *speedup reported in this section refers to the additional acceleration obtained when the framework's reconnaissance layer guides MCCO's deployment (2 chains instead of 3), measured against MCCO with default configuration—a comparison between guided and unguided deployment of the same algorithm. The two speedups address orthogonal sources of waste: the* $24{,}000\times$ *figure quantifies the benefit of selecting the right* algorithm family*; the* $3.56\times$ *figure quantifies the benefit of selecting the right* configuration *within that family. They are reported separately because they were measured against different baselines and should be interpreted independently.*

兩個加速倍數來自不同比較基準，不可混淆。Sub-box 6a 中 MCCO 的 $24{,}000\times$ *加速是 MCCO 相對於 GA+MC 基準的核心演算法加速——同一問題上兩種不同演算法的比較。本節的* $3.56\times$ *加速是在本框架偵察層引導下 MCCO 部署配置調整（由 3 條鏈降為 2 條鏈）相對於 MCCO 預設配置的額外加速——同一演算法下引導部署與盲目部署的比較。兩個加速指向正交的浪費來源：*$24{,}000\times$ *量化選對演算法家族的效益，*$3.56\times$ *量化在該家族內選對配置的效益。兩者使用不同基準，獨立報告，獨立解讀。*

12.2 Inverse-Variance Weighting Result

On the same test day, inverse-variance weighting across future time steps revealed a counter-intuitive weight distribution: the fifth hour ($t = 5$) received the highest weight (21.3%) while the first hour ($t = 1$) received only 9.4%, suggesting that the most informative forecast horizon is not necessarily the nearest one.

12.3 Implications for Other Boxes

The success of MCCO in Box 6 suggests a general strategy for unifying each box: identify a *mathematical substrate*—analogous to the collapse chain—that provides structural advantages (analytic gradients, automatic stability, bounded trajectories) across all sub-problems within the box. Potential substrates include:

Box	Potential Unification Substrate
1 (Combinatorial)	LP relaxation (integrality $\rightarrow$ continuity)
2 (Graph)	Laplacian matrix / spectral methods
3 (Logic)	Continuous relaxation (hardest; phase transition barrier)
4 (Path/Network)	LP + Column Generation
5 (Permutation)	Sinkhorn operator (permutation $\rightarrow$ doubly stochastic matrix)
6 (Sequential)	Collapse chain (demonstrated)

Each of these represents a research programme of comparable scope to the NSD/MOEH programme that produced MCCO.

13 Worked Example: TSP with Framework vs Blind Deployment

To demonstrate the framework's operational flow, we walk through a complete 10-customer Travelling Salesman Problem, comparing three approaches: (A) brute force, (B) blind deployment of the default algorithm, and (C) framework-guided deployment.

13.1 Problem Instance

A delivery company has one warehouse at coordinates $(50, 50)$ and 10 customers at random locations in a 100×100 grid. The objective is to find the shortest route visiting all customers and returning to the warehouse.

13.2 Framework Execution (Ten-Step Pipeline)

Step 1 (Classify): The problem is an ordering problem $\rightarrow$ Box 1, Sub-box 1b (TSP). Default algorithm: GA + 2-opt.

Step 2 (Construction): Swap neighbourhood (exchange two cities in the route).

Step 3 (Aerial survey): Spatial distribution coefficient of variation CV $= 0.398$ (moderately spread). Suggests moderate multimodality.

Step 4 (Scout sampling): 50 random permutations evaluated in < 0.001 s. Loss range: $[431.7, 646.7]$, mean $= 552.5$.

Step 5 (Scoring):

- Ruggedness (autocorrelation): 0.603 (moderate-smooth).
- Multimodality: 3 basins from 20-start local search (low).
- Feasibility: 100% (no constraints).
- Overall difficulty score: 35/100 (LOW).

Step 6 (Transform decision): Score $35 < 60$ threshold. No transformation needed.

Step 7 (Algorithm selection): Low difficulty with few basins → simple 2-opt local search suffices. GA is unnecessary for this landscape. *The framework downgrades from the default GA+2-opt to pure 2-opt.*

Step 8 (Parameter configuration): Multi-start with 10 random initialisations; take best result.

Step 9 (Solve): Runtime: 0.006 s. Best route distance: 322.14.

Step 10 (Refine): Not needed (solved on original landscape, no transformation applied).

13.3 Three-Way Comparison

Metric	Brute Force	Blind Default	Framework-Guided
Algorithm	Enumerate all	GA + 2-opt	2-opt only
Evaluations	3,628,800	$\sim$5,000	$\sim$100
Recon cost	—	—	< 0.001 s
Solve time	minutes	$\sim$1 s	0.006 s
Solution quality	optimal (guaranteed)	near-optimal	near-optimal (322.14)

13.4 Key Observations

Three findings emerge from this example:

1. **Framework-guided is fastest.** By recognising that the landscape is low-difficulty, the framework avoids deploying the heavier default algorithm (GA + 2-opt) and instead uses lightweight 2-opt with multi-start. The reconnaissance cost (< 0.001 s) is negligible compared to the time saved.

2. **Blind default is wasteful.** GA + 2-opt is the correct default when no landscape information is available. But for this instance, the GA's population management and crossover operations add overhead without improving the solution, because the landscape has only 3 basins—easily found by multi-start alone.

3. **Scout best vs final best.** The 50 scout samples found a best route of 431.7. The solver improved this to 322.14—a 25.4% reduction. This confirms that scouts are for *reconnaissance*, not for solving; their value lies in the landscape information they provide, not in the solutions they find.

13.5 Scaling Test: 20 and 50 Cities

To test scalability, the same framework was applied to larger instances:

Cities	Framework	Blind Default	Brute Force	Speedup
10	322.14 (0.006 s)	322.14 (1.0 s)	3.6×10^6 perms	166×
20	407.25 (1.8 s)	407.25 (3.1 s)	2.4×10^{18} perms	1.7×
50	573.78 (39.7 s)	573.16 (55.0 s)	3×10^{64} perms	1.4×

In all cases, the framework-guided solution matches or nearly matches the blind default in quality while completing faster. The reconnaissance overhead is negligible relative to the solving time.

13.6 How Close to P?

The most striking observation is the computational order of magnitude:

Method	Operations (50 cities)	Time
Polynomial $O(n^3)$	$\sim 10^5$	instantaneous
Framework-guided	$\sim 10^6$	**40 seconds**
Brute force $O(n!)$	$\sim 10^{64}$	$\sim 10^{38}$ universe lifetimes

The framework operates at $\sim 10^6$ evaluations—one order of magnitude above polynomial time (10^5) and *fifty-eight orders of magnitude* below brute force (10^{64}). In computational terms, the framework is already standing next to P. The brute force approach would require approximately 10^{38} times the age of the universe to complete; the framework finishes in under a minute.

This demonstrates the paper's central thesis in concrete terms: the mathematical distinction between P and NP persists, but the engineering distinction—as measured by actual wall-clock time on typical instances—can be substantially narrowed for this problem class.

Cross-box comparison. Extending this analysis to all six boxes reveals that several are already operating at polynomial time:

Box	Problem	Framework	P baseline	Gap to P
1a	MKP (DP feasible)	$O(NW)$	$O(NW)$	**= P** (pseudo-poly)
1b	TSP (50 cities)	$\sim 10^6$	$\sim 10^5$	1 order
2a	Graph Coloring	$O(N^2 \cdot \text{iter})$	$O(N^2)$	1–2 orders
3a	SAT ($\alpha \ll 4.26$)	$O(N)$	$O(N)$	**= P**
3a	SAT ($\alpha \approx 4.26$)	$O(N^{1-3})$ via CDCL	$O(N)$	$\approx$ **P** (CDCL)
4b	VRP (cluster+NN)	$O(N^2)$	$O(N^2)$	**= P**
5a	Scheduling (LPT)	$O(N \log N)$	$O(N \log N)$	**= P**
6a	MCCO	$\sim 10^4$	$\sim 10^3$	1 order

Five of eight test cases already operate at or near polynomial time complexity (four exactly polynomial, one via CDCL). The remaining three are within one to two orders of magnitude. No test case exhibits a "large gap" to P. SAT near the phase transition, previously the framework's weakest point, is effectively solved by CDCL—a state-of-the-art algorithm that the framework selects automatically when $\alpha > 3.0$. The framework achieves near-polynomial-time *approximate* solutions, not polynomial-time *exact* solutions—but the approximation quality (0.1–6.8% gap to optimal) is sufficient for virtually all practical applications.

八個測試案例中五個已達到或接近多項式時間複雜度（四個精確多項式，一個透過CDCL）。其餘三個只差一到兩個數量級。沒有任何一個案例存在「巨大差距」。框架不

發明新演算法，而是從現有最佳演算法中選對的那個。數學家要求的是 100% 精確最優 → NP 仍然是 NP。工程師要求的是 99%+ 最優在秒級完成 → 框架已經做到。

框架的運算量約 10^6，僅比多項式時間（10^5）高一個數量級，卻比暴力法（10^{64}）低五十八個數量級。框架已經站在 P 的旁邊。暴力法需要約 10^{38} 個宇宙壽命才能完成；框架在一分鐘內結束。數學上的 P 與 NP 區別仍在，但工程上的區別——以實際時鐘時間衡量——已經消失。

此範例展示了框架的核心價值：偵察發現曲面簡單 → 自動降級到最輕量演算法 → 不浪費算力。盲目使用預設演算法（GA+2-opt）在此實例上反而更慢。偵察成本可忽略，但節省的計算量是顯著的。斥候的價值在於提供地形資訊，而非提供解本身——斥候最佳路線 431.7 被求解器改善到 322.14，提升 25.4%。

13.7 TSPLIB Benchmark Validation

To provide rigorous empirical evidence, we tested the framework on three standard instances from TSPLIB [Reinelt, 1991], the reference benchmark used by virtually all TSP research. Each instance has a known optimal solution computed by the Concorde exact solver. We compare: (A) blind deployment (nearest neighbour + basic 2-opt, no reconnaissance), and (B) framework-guided deployment (reconnaissance → algorithm selection → matched solver). Both methods use the same tool family (NN + 2-opt); the only difference is whether reconnaissance informs the deployment.

Equal time budget comparison. To ensure fairness, both methods receive identical time budgets. The framework spends 4–25% of the budget on reconnaissance and uses the remainder for solving with its matched configuration.

Instance	N	Optimal	Blind (gap)	Framework (gap)	Recon cost
eil51	51	426	448 (5.2%)	**445 (4.5%)**	4% of budget
berlin52	52	7,542	7,804 (3.5%)	**7,685 (1.9%)**	5% of budget
kroA100	100	21,282	23,605 (10.9%)	**22,719 (6.8%)**	25% of budget

The framework outperforms blind deployment on all three standard benchmarks, with the largest improvement on kroA100 (4.1 percentage points closer to optimal). The reconnaissance cost is a small fraction of the total time budget, and the remaining time is used more efficiently due to matched algorithm configuration.

Key finding: On kroA100, the framework achieved a 6.8% gap to optimal versus the blind method's 10.9% gap—*with the same tools, in the same time*. The only difference was that the framework knew what the mountain looked like before climbing.

在 TSPLIB 標準基準測試上，框架在三個實例上均優於盲目部署。最大改善出現在 kroA100（偏離最優解從 10.9% 降至 6.8%）。偵察成本僅佔時間預算的 4–25%。同樣的工具、同樣的時間，唯一的差別是框架在爬山之前知道山的形狀。

13.8 Multidimensional Knapsack Benchmark (Sub-box 1a)

To validate the framework on a second sub-box, we tested on multidimensional knapsack problem (MKP) instances [Chu & Beasley, 1998]—a direct model of real-world logistics scenarios where packages must be selected subject to multiple constraints (weight, volume, fragility, temperature, priority). Unlike one-dimensional knapsack where greedy is near-optimal, MKP with multiple tight constraints creates a rugged, constrained landscape where greedy frequently selects suboptimal items.

We compare: (A) blind deployment (greedy by pseudo-utility ratio, no reconnaissance), and (B) framework-guided deployment (reconnaissance determines constraint difficulty, selects GA with repair operator seeded by greedy solution).

Instance	N	M	**Blind**	**Framework**	**Improvement**
50 packages, 3 constraints	50	3	997	**1,066**	+6.9%
100 packages, 5 constraints	100	5	2,184	**2,214**	+1.4%
200 packages, 10 constraints	200	10	4,546	4,546	0.0%
500 packages, 10 constraints	500	10	10,767	**11,067**	+2.8%

Key finding: The framework's advantage emerges specifically when constraints are multiple and tight. On the simplest instance (50 packages, 3 constraints), the GA with repair operator found a solution 6.9% better than greedy—69 additional units of value from the same set of packages. The reconnaissance system correctly identified that MKP is not solvable by DP (multidimensional) and selected GA with greedy-seeded initialisation and a constraint-aware repair operator.

This contrasts with one-dimensional knapsack, where greedy is near-optimal and the framework's value lies in identifying DP feasibility rather than algorithm upgrading. The framework adapts its strategy to the problem's constraint structure: simple constraints → DP or greedy; multiple constraints → GA with repair.

框架的優勢在多約束情境下特別明顯。50 包裹 3 約束時，GA + 修復算子比貪心找到高 6.9% 的方案——同一批包裹多裝了 69 單位價值。偵察正確判斷 MKP 無法用 DP 求解，選擇了以貪心初始化的 GA + 約束感知修復算子。這與一維背包形成對比：一維時貪心就夠好，框架的價值在於判斷 DP 可行性；多維時貪心不夠好，框架的價值在於升級演算法。

13.9 Graph Coloring Benchmark (Sub-box 2a)

To validate the framework across box boundaries, we tested on graph coloring—a fundamentally different problem class (Box 2, Graph Theory) from TSP and knapsack (Box 1, Combinatorial Optimisation). The objective is to assign colors to nodes such that no adjacent nodes share a color, using the minimum number of colors.

Graph coloring is highly sensitive to algorithm choice: basic greedy coloring depends critically on node ordering, and poor ordering wastes colors. The framework's reconnaissance analyses graph density and selects the appropriate algorithm: sparse graphs → degree-ordered greedy; medium density → DSatur [Brélaz, 1979] (saturation-based ordering); dense graphs → DSatur with Tabu Search refinement.

We compare: (A) blind deployment (basic greedy with arbitrary ordering), and (B) framework-guided deployment (reconnaissance-based algorithm selection).

Instance	N	**Density**	**Blind**	**Framework**	**Saved**
Sparse-50	50	0.11	5	**4**	1 (20%)
Medium-50	50	0.31	10	**7**	3 (30%)
Dense-50	50	0.49	12	**11**	1 (8%)
Sparse-100	100	0.11	7	**6**	1 (14%)
Medium-100	100	0.31	14	**12**	2 (14%)
Dense-100	100	0.50	21	**15**	6 (29%)
Dense-200	200	0.50	37	**26**	11 (30%)

Key finding: The framework reduces color count by 14–30% across all instances, with the largest improvements on dense graphs. On Dense-200, the framework uses 26 colors where blind greedy requires 37—11 fewer colors, a 30% reduction. The reconnaissance system correctly identified high density (> 0.5) as triggering Tabu Search refinement: DSatur provides a good initial coloring, then Tabu iteratively attempts to reduce the color count by reassigning conflicting nodes.

This result is particularly significant because it crosses box boundaries: the same framework that selects 2-opt for TSP (Box 1b) and GA for MKP (Box 1a) also selects DSatur + Tabu for graph coloring (Box 2a). The reconnaissance indicators are different (spatial CV for TSP, constraint count for MKP, graph density for coloring), but the meta-logic is identical: observe the landscape, classify the difficulty, match the algorithm.

框架在圖著色上的改善幅度（14–30%）甚至超過 TSP（4–8%）和 MKP（1.4–6.9%）。200 節點密集圖上，框架用 26 色而盲目需要 37 色——省 11 色。此結果跨越盒子邊界：同一框架為 TSP 選 2-opt、為 MKP 選 GA、為圖著色選 DSatur + Tabu。偵察指標不同（TSP 看空間分佈、MKP 看約束數量、圖著色看密度），但元邏輯相同：觀察曲面、分類難度、匹配演算法。

13.10 3-SAT Benchmark (Sub-box 3a)

To extend validation to the most theoretically significant box, we tested on 3-SAT—the canonical NP-complete problem with a well-known phase transition [Mézard et al., 2002] at clause-to-variable ratio $\alpha \approx 4.26$. Below this threshold, instances are almost certainly satisfiable; above it, almost certainly unsatisfiable; near it, instances are extremely hard.

The aerial survey indicator for Box 3 is trivially computed: $\alpha = M/N$ (number of clauses divided by number of variables). This single number, available at zero cost, determines the landscape difficulty and the optimal algorithm. The framework's algorithm selection for Box 3 is:

- $\alpha < 3.0$: WalkSAT [Selman et al., 1994] (random walk, fast on underconstrained instances).
- $3.0 \leq \alpha < 4.5$: CDCL (Conflict-Driven Clause Learning) [Marques-Silva & Sakallah, 1999, Een & Sörensson, 2003]—the state-of-the-art systematic SAT solver. Modern implementations (MiniSat, CaDiCaL, Glucose) solve instances with hundreds of thousands of variables in seconds.
- $\alpha > 4.5$: CDCL in proof mode (prove unsatisfiability).

We tested on instances with 50–200 variables across the phase transition, comparing: (A) blind deployment (WalkSAT with default parameters for all α values), and (B) framework-guided deployment (reconnaissance selects WalkSAT or CDCL based on α).

Instance	N	α	**Blind**	**Framework**	**Finding**
Easy-50	50	2.0	100/100 ✓	100/100 ✓	FW 1.5× faster
Medium-50	50	3.5	175/175 ✓	175/175 ✓	Tie
Hard-50	50	4.2	210/210 ✓	210/210 ✓	FW **15.1**× **faster**
Unsat-50	50	5.0	249/250 ×	detected unsat	FW avoids wasted effort
Easy-100	100	2.0	200/200 ✓	200/200 ✓	Tie
Medium-100	100	3.5	350/350 ✓	350/350 ✓	FW 1.4× faster
Hard-200	200	4.2	838/840 ×	**840/840** ✓	**FW solved; blind failed**

Key findings:

1. **Hard-50 ($\alpha = 4.2$):** The framework selected DPLL/CDCL and solved the instance $15.1\times$ faster than blind WalkSAT. Near the phase transition, systematic search dramatically outperforms random walk.

2. **Hard-200 ($\alpha = 4.2$):** Blind WalkSAT failed to satisfy all clauses (838/840), while the framework's systematic solver achieved full satisfaction (840/840). *The framework solved a problem that the blind method could not.*

3. **Unsat-50 ($\alpha = 5.0$):** The framework correctly identified $\alpha > 4.5$ as likely unsatisfiable and switched to proof mode, rather than wasting time on random walk.

The reconnaissance for Box 3 is the simplest of all boxes: a single division (M/N) at zero cost. Yet this single number determines whether the instance is easy, hard, or unsatisfiable, and which algorithm family is appropriate. This exemplifies the framework's core value: **the framework does not invent new algorithms; it selects the best existing algorithm for the problem at hand**. For Box 3, state-of-the-art CDCL solvers (MiniSat, CaDiCaL) can handle millions of variables—the framework's contribution is knowing *when* to deploy them versus simpler methods.

盒子 3 的偵察是所有盒子中最簡單的：一次除法（M/N），零成本。但這一個數字就決定了問題是簡單、困難還是不可滿足，以及該用哪種演算法。框架不發明新演算法，而是從現有最佳演算法中選對的那個。盒子 3 的最佳演算法是 CDCL——現代實作可處理數百萬變數。框架的貢獻是知道*何時*部署 CDCL 而非簡單的隨機搜索。

13.11 Vehicle Routing Benchmark (Sub-box 4b)

To validate the framework's cutting mechanism, we tested on the Vehicle Routing Problem (VRP) [Solomon, 1987]—a problem that naturally decomposes into "one sub-problem per vehicle." VRP is a direct model of logistics operations: a fleet of capacitated vehicles must serve all customers from a central depot at minimum total distance.

We compare: (A) blind deployment (nearest-neighbour per vehicle, sequential allocation), and (B) framework-guided deployment (cluster-first route-second: reconnaissance identifies geographic clusters, assigns each vehicle to a zone, then optimises each zone independently with NN + 2-opt).

Instance	N	Vehicles	Blind served	FW served	Additional
20 customers, 3 vehicles	20	3	12/20	**20/20**	+8
50 customers, 5 vehicles	50	5	30/50	**50/50**	+20
100 customers, 8 vehicles	100	8	56/100	**100/100**	+44
50 customers (clustered)	50	5	26/50	**50/50**	+24

Key finding: The framework's advantage in VRP is qualitatively different from TSP, MKP, and graph coloring. In those problems, the framework finds *better* solutions. In VRP, the framework serves *more customers*—a difference between partial and complete service. The blind method allocates customers to vehicles sequentially: the first few vehicles fill up, and remaining customers are left unserved. The framework's cluster-first approach distributes customers geographically across vehicles before routing, ensuring complete coverage.

This is the framework's **cutting mechanism** (Section 9) applied naturally: VRP decomposes into "one cluster per vehicle," each cluster is routed independently, and the result covers

all customers. The reconnaissance (spatial CV) identifies whether customers are clustered or uniformly distributed, informing the clustering strategy.

Remark 13 (On global refinement for VRP)**.** *Section 9 notes that cutting without a global refinement step degrades boundary-adjacent solution quality by 7–11% on length-optimisation tasks such as TSP, because swap moves near cut boundaries may yield additional improvements. For the VRP experiments reported above, the objective of interest is customer coverage (a feasibility-dominated metric), not total route length; cluster-and-route is sufficient to achieve the reported 100% coverage results without a boundary-refinement pass. For applications where both coverage and route length minimisation are required, a global refinement step—2-opt or Or-opt across boundary edges between clusters, as in Section 9—should be applied after the per-cluster route optimisation.*

框架在 VRP 上的優勢與 TSP、MKP、圖著色質性不同。前三者中，框架找到*更好的解*；在 VRP 中，框架服務*更多客戶*——部分服務與完整服務的差別。盲目方法按順序分配客戶給車輛，前幾輛車裝滿後剩餘客戶無人服務。框架的分群先行法在排路線之前先按地理分配客戶，確保完全覆蓋。這正是框架切割機制（Section 9）的天然應用：每台車一個子問題。Section 9 的 7–11% boundary 退化觀察針對長度最佳化；此處 VRP 實驗的目標為客戶覆蓋率（可行性主導），分群即可達成 100% 覆蓋。若同時要求覆蓋與路線長度最佳，則需在每群路線最佳化後加上跨邊界 2-opt / Or-opt 全局精修。

13.12 Parallel Machine Scheduling Benchmark (Sub-box 5a)

To complete the cross-box validation, we tested on parallel machine scheduling—assigning N jobs with varying processing times to M identical machines to minimise the makespan (completion time of the last machine). This is a core problem in manufacturing, cloud computing, and logistics.

We compare: (A) blind deployment (round-robin assignment, ignoring job sizes), and (B) framework-guided deployment (reconnaissance detects job size distribution, selects LPT [Graham, 1969]—Longest Processing Time first—with optional SA refinement for difficult instances).

Instance	N	M	**LB**	**Blind**	**Framework**	**Improvement**
20 jobs, 3 machines	20	3	213	232	**214**	+7.8%
50 jobs, 5 machines	50	5	290	313	**291**	+7.0%
50 jobs, variable	50	5	502	569	**503**	+11.6%
200 jobs, 10 machines	200	10	987	1,244	**988**	+20.6%
100 jobs, bimodal	100	3	1,269	1,739	**1,272**	+26.9%
50 jobs, 10 machines	50	10	251	341	**254**	+25.5%

Key findings:

1. **Near-optimal solutions:** On the 200-job instance, the framework found a makespan of 988 against a lower bound of 987—only 0.1% from optimal. Blind round-robin was 26% above optimal.

2. **Improvement accelerates with scale:** 7.8% at 20 jobs $\rightarrow$ 11.6% at 50 jobs $\rightarrow$ 20.6% at 200 jobs $\rightarrow$ 26.9% at 100 bimodal jobs. The larger and more variable the problem, the more blind allocation wastes capacity.

3. **Reconnaissance adapts strategy:** Low job-time variance → LPT alone suffices. High variance or tight capacity → LPT + SA refinement. The framework does not over-invest in easy problems.

The scheduling results exhibit the same accelerating-improvement pattern seen in TSP (Section 13): framework advantage grows super-linearly with problem difficulty. Round-robin allocation is the scheduling equivalent of "climbing without looking"—it ignores the most basic landscape feature (job size distribution) and pays a heavy penalty.

排班結果展現與 TSP 相同的加速改善模式：框架優勢隨問題難度超線性增長。200 工作 10 機器時，框架找到的 makespan（988）僅比下界（987）高 0.1%，而盲目分配偏離 26%。偵察根據工作時間分佈自動選擇：變異低 → 純 LPT；變異高 →LPT + SA 精修。

13.13 Improvement Scales with Problem Size

A striking pattern emerges from the scaling tests: the framework's advantage over blind deployment *increases* with problem size.

Problem Size	Framework Improvement	Trend
10 cities	25.4% (route quality)	—
20 cities	9.1% (matrix vs blind)	baseline
50 cities	65.8%	sharp increase
100 cities	80.3%	continues rising

This acceleration is not linear—it is super-linear. The larger the problem, the more computation blind methods waste on mismatched algorithms, and the more the framework saves by informed deployment.

13.14 Scalability Validation (1,000+ Instances)

To address the concern that the framework's advantages might diminish at industrial scale, we tested on larger instances:

TSP (1,000 cities): The framework (20-start NN + tuned 2-opt) achieved a route length of 27,333 versus blind (single NN + basic 2-opt) at 28,334—a 3.5% improvement. While smaller in percentage than the 100-city result (8.3%), this reflects the limited 2-opt tool, not the framework. With stronger tools (e.g., LKH [Helsgaun, 2000]), the framework's advantage would compound further.

Graph Coloring (DIMACS-style, 125–500 nodes): Following the DIMACS benchmark convention [Johnson et al., 1996], we generated random graphs at densities matching standard test instances:

Instance	N	Edges	Blind	Framework	Saved
DSJC125.5-like	125	3,903	26	**23**	3 (12%)
DSJC250.5-like	250	15,541	40	**37**	3 (8%)
DSJC500.1-like	500	12,541	19	**16**	3 (16%)

The framework's advantage persists at 500 nodes, confirming scalability across problem types.

1,000 城市 TSP 和 500 節點圖著色的測試結果確認框架的優勢在大規模實例上持續存在。

14 Complexity Reference

Sub-box	Best-Fit Solver	Complexity	Typical Time
1a Knapsack	DP	$O(NW)$	seconds
1b TSP	GA + 2-opt	$O(G \cdot P \cdot N^2)$	minutes–hours
1c Bin Packing	SA / FFD	$O(N \log N)$–$O(\text{iter} \cdot N)$	seconds–minutes
1d Cutting Stock	Column Generation	$O(\text{iter} \cdot \text{subproblem})$	minutes
2a Colouring	Tabu Search	$O(\text{iter} \cdot \lvert V \rvert)$	minutes–hours
2b Vertex Cover	Greedy 2-approx	$O(V + E)$	seconds
2c Maximum Clique	Branch & Bound / GA	exponential	hours–days
2d Steiner Tree	Approximation	$O(\lvert V \rvert^2)$–$O(\lvert V \rvert^3)$	seconds–minutes
3a 3-SAT	CDCL / WalkSAT	variable	seconds–hours
3b CSP	Backtracking + AC	variable	seconds–hours
3c MAX-SAT	SA / B&B	variable	minutes–hours
4a Hamilton Path	GA + local search	$O(G \cdot P \cdot N)$	minutes
4b VRP	LNS / ACO	$O(\text{iter} \cdot N^2)$	minutes–hours
4c Facility Location	LP + B&B	$O(\text{LP iter} \cdot NM)$	minutes–hours
5a Nurse Scheduling	CP / Tabu	variable	minutes–hours
5b Matching	Hungarian	$O(N^3)$	seconds
5c QAP	SA / large GA	$O(G \cdot P \cdot N^2)$	hours–days
6a Financial (MCCO)	MCCO	$O(C \cdot M^* \cdot E \cdot T) \approx 10^4$	**41 seconds**
6b Medical	MCCO variant	$\approx 10^4$–10^5	minutes
6c Control	MCCO / RL	variable	minutes–hours

15 Known Challenges and Preventive Measures

Each algorithm family faces known failure modes. A key contribution of this framework is to enumerate these in advance, enabling preventive design rather than reactive debugging.

Algorithm	Known Problem	Prevention
Gradient descent	Vanishing/exploding gradients	Chain Forgetting ($\rho < 1$); gradient clipping
GA	Premature convergence	Adaptive mutation rate; age-limited elite
Simulated Annealing	Cooling schedule sensitivity	Adaptive cooling; reheating
Tabu Search	Tabu list size sensitivity	Adaptive list length
Local Search	Stuck in local optima	Multi-start; perturbation after convergence
Branch & Bound	Exponential worst case	Improved bounds; problem-specific cuts
CDCL (SAT)	Memory explosion (clause learning)	Clause deletion strategies
Constraint Programming	Search tree explosion	Improved propagation; symmetry breaking
LP relaxation	Integrality gap	Cutting planes; branch and cut

16 Discussion

16.1 Relation to Existing Work

The proposed framework builds upon, integrates, and extends six existing lines of research. We distinguish each clearly to establish the novelty of the present contribution.

16.1.1 Classical Problem Taxonomy

The foundational classification of NP-complete problems is due to Garey & Johnson [1979], who catalogued hundreds of NP-complete problems organised by application domain (graph theory, scheduling, mathematical programming, logic, etc.). This taxonomy remains the standard reference and is the starting point for any study of NP-hardness. However, it is purely descriptive: it tells the practitioner that a problem is NP-complete, but provides no guidance on how to solve it, which algorithm to use, or what the loss landscape looks like. Our framework takes the Garey–Johnson catalogue as input (Box assignment) and adds three operational layers on top: reconnaissance, landscape classification, and algorithm-parameter matching.

Garey–Johnson 的分類是「問題叫什麼名字」的名冊。本框架是「問題怎麼解」的導航系統。前者是起點，後者是在其基礎上往前推進的完整操作流程。

16.1.2 Fitness Landscape Analysis (FLA) and Exploratory Landscape Analysis (ELA)

Fitness Landscape Analysis [Pitzer & Affenzeller, 2012] and its quantitative extension, Exploratory Landscape Analysis [Mersmann et al., 2011], analyse individual problem landscapes by computing statistical features such as multimodality, ruggedness, basin size, and local optima networks. This line of work provides the analytical vocabulary that our framework adopts (unimodal, multimodal, rugged, deceptive, plateau). However, FLA/ELA studies typically analyse one problem class at a time and do not provide a cross-problem taxonomy or an end-to-end algorithm selection pipeline. Our framework is a natural extension and operationalisation of FLA: we take FLA features as the basis for a universal classification system, add a three-tier reconnaissance mechanism to acquire these features efficiently, and map them directly to algorithm-parameter configurations.

本框架是 FLA 的自然延伸與升級版——將 FLA 特徵直接轉化為七盒子分類 + 可執行的參數配置表。

16.1.3 Algorithm Selection Problem

The Algorithm Selection Problem, formalised by Rice [1976], proposes selecting algorithms based on measurable problem features. Modern implementations [Kotthoff, 2016] use machine learning to predict which solver will perform best on a given instance. Prominent systems include SATzilla [Xu et al., 2008], which builds per-instance runtime prediction models for SAT solvers using 48+ problem features, and AutoFolio [Lindauer et al., 2015], which automatically configures algorithm selection systems using algorithm configuration techniques. More recent work by Kerschke & Trautmann [2019] extends ELA-based selection to continuous optimisation.

These systems have demonstrated strong practical results, particularly for SAT and combinatorial auctions. However, they differ from our framework in three key aspects: (1) they require extensive training data (thousands of solved instances per problem class), whereas our framework uses zero training data—the reconnaissance is computed per-instance at negligible cost; (2) they operate as black boxes (the learned model does not explain *why* a particular algorithm

is suited), whereas our framework provides transparent, topology-based reasoning; (3) they are specialised per-domain (a SAT selector cannot select VRP algorithms), whereas our framework spans all six boxes with a unified decision tree.

Our framework can be viewed as a complementary approach: where SATzilla/AutoFolio use data-driven prediction, we use structure-driven classification. In domains with abundant training data, ML-based selection may achieve finer granularity; in domains with limited data or novel problem types, our framework provides a principled starting point.

16.1.4 No Free Lunch Theorem

The No Free Lunch Theorem [Wolpert & Macready, 1997] proves that no single optimisation algorithm can dominate all others across all possible problems. This result is often cited as a fundamental barrier to universal algorithm design. Our framework does not contradict No Free Lunch; rather, it provides the constructive answer that NFL left open. We do not propose a universal *algorithm*, but a universal *selection framework*—a meta-algorithm that determines which specialised algorithm to deploy for each landscape type.

16.1.5 Parameterized Complexity

Parameterized Complexity [Downey & Fellows, 2013] analyses NP-hard problems by identifying structural parameters (treewidth, vertex cover number, solution size) that, when bounded, render the problem solvable in FPT (Fixed-Parameter Tractable) time. This provides a fine-grained theoretical complement to our framework: sub-boxes where the relevant parameter is bounded can be marked as "easy" (unimodal or smooth landscape), while sub-boxes where the parameter is unbounded correspond to rugged or deceptive landscapes. Incorporating parameterized indicators into our Tier 1 aerial survey would strengthen the theoretical grounding of the reconnaissance system.

參數化複雜度為本框架提供理論補強：參數有界的子盒子對應光滑曲面，參數無界的對應崎嶇曲面。

16.1.6 Learning-Augmented Algorithms

A recent line of work [Bampis et al., 2025] explores algorithms that receive predictions from an external oracle (e.g., "is element i before element j in the optimal permutation?"). When predictions are accurate, these algorithms achieve polynomial-time performance on NP-hard permutation problems; when predictions are poor, they degrade gracefully to standard worst-case bounds. This paradigm is complementary to our framework: our reconnaissance system can be viewed as a self-contained prediction oracle that estimates landscape properties without external input. Future integration of learning-augmented techniques with our Tier 2 scout sampling could further accelerate algorithm deployment.

16.1.7 Summary

Framework	Actionable	Interpretable	Universal	Relation to Ours
Garey–Johnson (1979)	low	high	high	Our input (Box assignment)
FLA / ELA	medium	high	medium	We extend and operationalise
Algorithm Selection (ML)	high	low	medium	We provide interpretable alternative
No Free Lunch	—	high	high	We provide constructive answer
Parameterized Complexity	low	high	medium	Theoretical complement
Learning-Augmented	high	medium	medium	Future integration
This paper	**high**	**high**	**high**	—

No existing framework simultaneously provides high actionability (end-to-end pipeline from problem to algorithm to parameters), high interpretability (landscape-based reasoning rather than black-box prediction), and high universality (coverage of all major structural types of NP-hard problems). The present framework is, to our knowledge, the first to achieve all three.

16.2 Limitations

1. **Sub-box boundaries are not always sharp.** Some problems exhibit properties of multiple sub-boxes simultaneously. A VRP with time windows and crew scheduling constraints spans Boxes 4 and 5. The current framework does not provide a formal cross-box decomposition mechanism; Box 7c (Hybrid) acknowledges this but does not resolve it.

2. **Landscape topology depends on construction method.** The same problem can yield different landscapes depending on the neighbourhood definition or relaxation method chosen. TSP with swap neighbourhood produces a different landscape than TSP with 2-opt neighbourhood. The framework recommends a default construction method per sub-box but acknowledges that the choice is not unique.

3. **Only Box 6 has a complete proof of concept.** The remaining five boxes have proposed unification substrates but no empirical validation. Each box requires a dedicated research programme comparable in scope to the NSD/MOEH programme that produced MCCO.

4. **The seven-box taxonomy may not be exhaustive.** Novel problem classes emerging from new application domains may require additional boxes.

5. **Transformation precision loss is difficult to quantify a priori.** Each landscape transformation incurs precision loss when mapping the solution back to the original space. LP relaxation has theoretical bounds on the integrality gap, but smoothing and embedding lack general-purpose precision loss estimates. The refinement step recovers some precision, but the recovery is not guaranteed to be complete.

6. **Transformation ordering is not specified.** When a landscape simultaneously exceeds multiple score thresholds (e.g., too rugged *and* too constrained), the framework does not prescribe whether to smooth first or relax constraints first. Different orderings may yield different transformed landscapes.

7. **Difficulty scores have non-trivial computational cost.** Among the four scores, multimodality requires multiple local search runs, and FDC requires a best-known solution

(which may not exist for novel instances). For very large-scale problems, the cost of computing these scores may itself be significant. In practice, when a true optimum is unavailable, we recommend substituting the *best-so-far solution* obtained during Tier 2 scout sampling as the reference anchor for FDC. This yields a biased but usable estimate: the correlation remains informative about local landscape geometry even if the anchor is not globally optimal, and the bias is bounded by the scout's gap to optimum. Users should report the anchor's provenance (true optimum vs. scout best-so-far) when citing FDC values.

8. **Excessive transformation may distort the problem.** After multiple rounds of aggressive transformation, the landscape being solved may bear little resemblance to the original problem. If the refinement step cannot bridge this gap, the returned solution may be of poor quality despite appearing optimal on the transformed landscape.

9. **The operational definition of Ruggedness (R) requires further refinement.** The proposed definition $R = 1-\rho_1$ based on lag-1 autocorrelation of a random walk (Section 3.1.1) is intuitively appealing but may have limited discriminating power across Box types when applied naively. Preliminary calculations on representative instances from Boxes 1, 2, and 3 suggest that the resulting R values cluster within a narrow range regardless of the underlying landscape's true ruggedness, because single-variable perturbations in structured neighbourhoods preserve high lag-1 correlation by construction. Candidate refinements include: (i) normalising by the neighbourhood-induced variance change rather than raw autocorrelation; (ii) adopting Weinberger's correlation length with variance-normalised moves [Pitzer & Affenzeller, 2012]; (iii) using information-content-based measures [Mersmann et al., 2011] in place of autocorrelation. A systematic cross-Box calibration of R remains future work; the indicator is retained in the current framework as a placeholder with the caveat that its operational form should be refined before being used as the sole basis for classification decisions.

   本命題的 R 指標運算化定義需要進一步改進。論文提出的 $R = 1 - \rho_1$（以隨機遊走的 lag-1 自相關為基礎）直觀上合理，但初步測試顯示，此定義在不同 Box 類別之間的區別力可能有限——因為單變數擾動在結構化鄰域中天然保留高 lag-1 相關性，導致 R 數值集中在狹窄範圍內，無法反映 landscape 的真實崎嶇度。候選改進方案包括：(i) 以鄰域誘導的變異數變化正規化，取代原始自相關；(ii) 採用 Weinberger 的 correlation length 搭配變異數正規化的移動；(iii) 改用 information-content 測度取代自相關。R 的系統性跨 Box 校準留作未來工作；本指標在當前框架中作為佔位符保留，但不應作為分類決策的唯一依據。

16.3 Not P = NP, But Practical Convergence

This framework does not claim to resolve the P $\stackrel{?}{=}$ NP question. However, if each box can be equipped with a structural substrate that provides analytic gradients or equivalent acceleration—as MCCO demonstrated for Box 6 with a $24{,}000\times$ speedup on the specific instances tested—then the practical computational gap between NP-hard problems and polynomial-time solutions may be substantially narrowed on classes of instances where such substrates exist. The mathematical distinction remains; the engineering distinction, in those regimes, becomes less operationally consequential.

本框架不主張解決 P $\stackrel{?}{=}$ NP 問題。但如果每個盒子都能找到提供結構性加速的數學基底——如 MCCO 在盒子六實現的 24,000 倍加速——則 NP 難題與多項式時間可解性之

間的實務差距可被縮小到操作上無關緊要的程度。數學上的區別仍在，工程上的區別消失。

16.4 An AI-Executable Protocol

A deliberate design choice of this framework is that every decision in the ten-step pipeline is table-driven and mechanisable. The seven-box taxonomy, the reconnaissance indicators, the difficulty scores, the transformation methods, the stopping conditions, the algorithm-matching table, and the parameter configuration table are all structured as lookup operations with explicit inputs and outputs. No step requires human intuition, domain expertise, or subjective judgement.

This makes the framework directly usable as an *AI-executable protocol*. A large language model or automated solver system, given this paper as input along with an NP-hard problem instance, can:

1. Parse the problem formulation and classify it into a box and sub-box.
2. Look up the aerial survey indicators for that box and compute them.
3. Determine whether scout sampling is applicable; if so, execute it.
4. Compute the four difficulty scores and check against thresholds.
5. If scores exceed thresholds, look up the transformation table, apply the appropriate transformation, and re-score.
6. Look up the algorithm-matching table and select the solver.
7. Look up the parameter configuration table and set parameters.
8. Execute the solver and return the solution.

The exhaustive enumeration of all 21 main sub-boxes (Boxes 1–6)—with default algorithms, scout-based adjustments, and landscape-based parameter settings—was designed precisely to enable this kind of mechanical execution. Human practitioners may find the tables verbose; automated systems find them exactly sufficient.

The practical implication is significant: any researcher or engineer who encounters an NP-hard problem can provide this paper and their problem description to an AI assistant, and receive a fully configured solution pipeline without needing to understand the underlying algorithm theory. The framework thus serves as a bridge between theoretical algorithm design and practical problem solving, mediated by AI systems.

本框架的每一步決策都是表格驅動且可機械化的。七盒子分類、偵察指標、難度分數、變形方法、停止條件、演算法配對表、參數配置表——全部結構化為明確的輸入輸出查詢操作，不需要人類直覺或主觀判斷。這使得本框架可直接作為 AI 可執行協議使用：將本論文和問題實例提供給大型語言模型或自動求解系統，即可獲得完整配置的求解流程。21 個主要子盒子（盒子 1–6）的窮舉式列舉——預設演算法、偵察調整、參數配置——正是為了實現這種機械化執行而設計的。任何遇到 NP 難題的研究者或工程師，只需將本論文和問題描述提供給 AI 助手，即可獲得完整的求解方案，無需理解底層演算法理論。

17 Conclusion

We have proposed a landscape classification framework for NP-hard problems consisting of seven boxes, 24 sub-boxes, a three-tier reconnaissance system, and a five-cut decision tree. The framework provides a systematic pipeline from problem identification to algorithm deployment, eliminating trial-and-error selection. Box 6 has been fully validated through MCCO, demonstrating that landscape-guided deployment achieves measurable computational savings with no loss in solution quality.

The framework's deepest implication is a shift in perspective: from "which algorithm should I try?" to "what does the mountain look like?" The former leads to trial and error. The latter leads to informed deployment. Just as database normalisation gave practitioners a finite procedure for structuring arbitrarily complex data, landscape classification gives practitioners a finite procedure for attacking arbitrarily complex optimisation problems.

The five remaining boxes represent five open research programmes, each seeking its own "collapse chain"—a mathematical substrate that converts structural complexity into computational efficiency.

Ultimately, the framework enables four layers of acceleration that compound multiplicatively:

1. **Reconnaissance**: selecting the right algorithm eliminates wasted computation on mismatched solvers.

2. **Transformation**: simplifying the landscape before solving reduces the intrinsic difficulty of the problem.

3. **Cutting**: decomposing large landscapes into parallel sub-problems reduces wall-clock time proportionally to the number of blocks.

4. **AI-executable protocol**: automating the entire decision pipeline from problem classification to parameter configuration reduces human decision time from days to seconds.

The four layers have been validated independently in this paper: reconnaissance via the TSPLIB, DIMACS-style, and Solomon VRP benchmarks (Section 13), transformation via the LP relaxation and smoothing discussion (Section 8), cutting via the VRP coverage results (Section 13), and AI-executable protocol via the table-driven design throughout Sections 6–7. A direct end-to-end measurement of the full compounded speedup across all four layers on a single benchmark instance remains future work.

本文對四層加速已分別獨立驗證：偵察層經由 TSPLIB、DIMACS 風格與 Solomon VRP 基準測試（Section 13）、變形層經由 LP 鬆弛與 smoothing 的討論（Section 8）、切割層經由 VRP 覆蓋率結果（Section 13）、AI 可執行協議層經由 Section 6–7 的表格化設計。四層在單一基準實例上端到端疊加加速的直接量測，留作未來工作。

Each layer independently reduces computational cost; together, they push the practical solving time of NP-hard problems progressively closer to polynomial time on instances within the framework's scope. Empirical results on TSPLIB benchmarks show that the framework's advantage over blind deployment grows with problem size (9% at 20 cities, 66% at 50 cities, 80% at 100 cities), though further testing at industrial scales (1000+ cities) is required to confirm whether the trend continues. The mathematical distinction between P and NP persists. The engineering distinction, under this framework, is substantially narrowed on the problem classes and scales tested.

本文提出了一個由七個盒子、24 個子盒子、三層偵察系統和五刀決策樹組成的 NP 難題曲面分類框架。框架提供從問題識別到演算法部署的系統化流程，消除試誤式選擇。最深層的含義是視角的轉換：從「該試哪個演算法」轉為「這座山長什麼樣」。剩餘五個盒子代表五個開放研究計畫，各自尋找自己的「坍縮鏈」。

框架實現四層乘法加速：偵察（選對演算法）× 變形（簡化曲面）× 切割（平行分解）× AI 執行協議（決策時間從天壓到秒）。TSPLIB 基準測試證實框架的優勢隨問題規模加速遞增（20 城市 9%→50 城市 66%→100 城市 80%）。P 與 NP 的數學區別可能永遠存在；在本框架下，工程區別被設計為消失。

References

Pitzer, E. and Affenzeller, M. A comprehensive survey on fitness landscape analysis. In *Recent Advances in Intelligent Engineering Systems*, pages 161–191. Springer, 2012.

Rice, J.R. The algorithm selection problem. *Advances in Computers*, 15:65–118, 1976.

Wolpert, D.H. and Macready, W.G. No free lunch theorems for optimization. *IEEE Transactions on Evolutionary Computation*, 1(1):67–82, 1997.

Mersmann, O., Bischl, B., Trautmann, H., Preuss, M., Weihs, C., and Rudolph, G. Exploratory landscape analysis. In *Proceedings of the 13th Annual Conference on Genetic and Evolutionary Computation (GECCO)*, pages 829–836, 2011.

Holland, J.H. *Adaptation in Natural and Artificial Systems*. University of Michigan Press, 1975.

Kirkpatrick, S., Gelatt, C.D., and Vecchi, M.P. Optimization by simulated annealing. *Science*, 220(4598):671–680, 1983.

Glover, F. Tabu search—Part I. *ORSA Journal on Computing*, 1(3):190–206, 1989.

Dorigo, M. and Gambardella, L.M. Ant colony system: A cooperative learning approach to the traveling salesman problem. *IEEE Transactions on Evolutionary Computation*, 1(1):53–66, 1997.

Moskewicz, M.W., Madigan, C.F., Zhao, Y., Zhang, L., and Malik, S. Chaff: Engineering an efficient SAT solver. In *Proceedings of the 38th Design Automation Conference*, pages 530–535, 2001.

Reinelt, G. TSPLIB—A traveling salesman problem library. *ORSA Journal on Computing*, 3(4):376–384, 1991.

Chu, P.C. and Beasley, J.E. A genetic algorithm for the multidimensional knapsack problem. *Journal of Heuristics*, 4:63–86, 1998.

Garey, M.R. and Johnson, D.S. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W.H. Freeman, 1979.

Kotthoff, L. Algorithm selection for combinatorial search problems: A survey. *AI Magazine*, 35(3):48–60, 2016.

Downey, R.G. and Fellows, M.R. *Fundamentals of Parameterized Complexity*. Springer, 2013.

Bampis, E., Escoffier, B., and Svensson, O. Learning-augmented algorithms for NP-hard permutation problems. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, 2025.

Brélaz, D. New methods to color the vertices of a graph. *Communications of the ACM*, 22(4):251–256, 1979.

Graham, R.L. Bounds on multiprocessing timing anomalies. *SIAM Journal on Applied Mathematics*, 17(2):416–429, 1969.

Selman, B., Kautz, H.A., and Cohen, B. Noise strategies for improving local search. In *Proceedings of the 12th National Conference on Artificial Intelligence (AAAI)*, pages 337–343, 1994.

Marques-Silva, J.P. and Sakallah, K.A. GRASP: A search algorithm for propositional satisfiability. *IEEE Transactions on Computers*, 48(5):506–521, 1999.

Xu, L., Hutter, F., Hoos, H.H., and Leyton-Brown, K. SATzilla: Portfolio-based algorithm selection for SAT. *Journal of Artificial Intelligence Research*, 32:565–606, 2008.

Lindauer, M., Hoos, H.H., Hutter, F., and Schaub, T. AutoFolio: An automatically configured algorithm selector. *Journal of Artificial Intelligence Research*, 53:745–778, 2015.

Mézard, M., Parisi, G., and Zecchina, R. Analytic and algorithmic solution of random satisfiability problems. *Science*, 297(5582):812–815, 2002.

Helsgaun, K. An effective implementation of the Lin–Kernighan heuristic. *European Journal of Operational Research*, 126(1):106–130, 2000.

Solomon, M.M. Algorithms for the vehicle routing and scheduling problems with time window constraints. *Operations Research*, 35(2):254–265, 1987.

Johnson, D.S. and Trick, M.A., editors. *Cliques, Coloring, and Satisfiability: Second DIMACS Implementation Challenge*. American Mathematical Society, 1996.

Eén, N. and Sörensson, N. An extensible SAT-solver. In *Theory and Applications of Satisfiability Testing (SAT 2003)*, LNCS 2919, pages 502–518, 2003.

Kerschke, P. and Trautmann, H. Automated algorithm selection on continuous black-box problems by combining exploratory landscape analysis and machine learning. *Evolutionary Computation*, 27(1):99–127, 2019.

tonyrao (饒 滙 盈). Memetic Collapse-Chain Optimisation: A Self-Adaptive Multi-Scale Framework for Decision Optimisation in Indeterminate Systems. *Zenodo Preprint*, DOI: 10.5281/zenodo.19487553, 2026.

tonyrao (饒滙盈). The Collapse Chain: Observation, Time, and the Structure of Indeterminate Systems. *Zenodo Preprint*, DOI: 10.5281/zenodo.19456757, 2026.

tonyrao (饒滙盈). From Prediction to Resilience: A Market Observation Equivalence Hypothesis. *Zenodo Preprint*, DOI: 10.5281/zenodo.19456940, 2026.

Pisinger, D. Where Are the Hard Knapsack Problems? *Computers & Operations Research*, 32(9):2271–2284, 2005.

Martello, S. and Toth, P. *Knapsack Problems: Algorithms and Computer Implementations*. John Wiley & Sons, 1990.

Applegate, D. L., Bixby, R. E., Chvátal, V., and Cook, W. J. *The Traveling Salesman Problem: A Computational Study*. Princeton University Press, 2006.

Coffman, E. G., Csirik, J., Galambos, G., Martello, S., and Vigo, D. Bin Packing Approximation Algorithms: Survey and Classification. In *Handbook of Combinatorial Optimization*, pp. 455–531. Springer, 2013.

Gilmore, P. C. and Gomory, R. E. A Linear Programming Approach to the Cutting-Stock Problem. *Operations Research*, 9(6):849–859, 1961.

Galinier, P. and Hertz, A. A Survey of Local Search Methods for Graph Coloring. *Computers & Operations Research*, 33(9):2547–2562, 2006.

Nemhauser, G. L. and Trotter, L. E. Vertex Packings: Structural Properties and Algorithms. *Mathematical Programming*, 8:232–248, 1975.

Chvátal, V. A Greedy Heuristic for the Set-Covering Problem. *Mathematics of Operations Research*, 4(3):233–235, 1979.

Dinur, I. and Steurer, D. Analytical Approach to Parallel Repetition. In *STOC 2014*, pp. 624–633. ACM, 2014.

Feige, U. A Threshold of $\ln n$ for Approximating Set Cover. *Journal of the ACM*, 45(4):634–652, 1998.

Cygan, M., Fomin, F. V., Kowalik, L., Lokshtanov, D., Marx, D., Pilipczuk, M., Pilipczuk, M., and Saurabh, S. *Parameterized Algorithms*. Springer, 2015.

Bomze, I. M., Budinich, M., Pardalos, P. M., and Pelillo, M. The Maximum Clique Problem. In *Handbook of Combinatorial Optimization*, pp. 1–74. Springer, 1999.

Hwang, F. K. and Richards, D. S. Steiner Tree Problems. *Networks*, 22(1):55–89, 1992.

Davis, M., Logemann, G., and Loveland, D. A Machine Program for Theorem-Proving. *Communications of the ACM*, 5(7):394–397, 1962.

Rossi, F., van Beek, P., and Walsh, T. *Handbook of Constraint Programming*. Elsevier, 2006.

Pinedo, M. L. *Scheduling: Theory, Algorithms, and Systems*. Springer, 5th edition, 2016.

Mladenovic, N., Drazic, M., Kovacevic-Vujcic, V., and Cangalovic, M. General Variable Neighborhood Search for the Continuous Optimization. *European Journal of Operational Research*, 191(3):753–770, 2007.

Toth, P. and Vigo, D. (Eds.). *Vehicle Routing: Problems, Methods, and Applications*. SIAM, 2nd edition, 2014.

Shaw, P. Using Constraint Programming and Local Search Methods to Solve Vehicle Routing Problems. In *CP 1998*, LNCS 1520, pp. 417–431. Springer, 1998.

Ahuja, R. K., Magnanti, T. L., and Orlin, J. B. *Network Flows: Theory, Algorithms, and Applications*. Prentice Hall, 1993.

Brandimarte, P. Routing and Scheduling in a Flexible Job Shop by Tabu Search. *Annals of Operations Research*, 41:157–183, 1993.

Kuhn, H. W. The Hungarian Method for the Assignment Problem. *Naval Research Logistics Quarterly*, 2(1–2):83–97, 1955.

Burkard, R., Dell'Amico, M., and Martello, S. *Assignment Problems*. SIAM, 2009.

Loiola, E. M., de Abreu, N. M. M., Boaventura-Netto, P. O., Hahn, P., and Querido, T. A Survey for the Quadratic Assignment Problem. *European Journal of Operational Research*, 176(2):657–690, 2007.

Sutton, R. S. and Barto, A. G. *Reinforcement Learning: An Introduction*. MIT Press, 2nd edition, 2018.

Disclaimer / 免責聲明

This working paper is provided for academic and informational purposes only.
本工作論文僅供學術和資訊目的。